

A Survey of Graph-Based Deep Learning for Anomaly Detection in Distributed Systems

Armin Danesh Pazho^{ID}, *Student Member, IEEE*, Ghazal Alinezhad Noghre^{ID}, *Student Member, IEEE*, Arnab A Purkayastha^{ID}, Jagannadh Vempati^{ID}, Otto Martin^{ID}, and Hamed Tabkhi^{ID}, *Member, IEEE*

Abstract—Anomaly detection is a crucial task in complex distributed systems. A thorough understanding of the requirements and challenges of anomaly detection is pivotal to the security of such systems, especially for real-world deployment. While there are many works and application domains that deal with this problem, few have attempted to provide an in-depth look at such systems. In this survey, we explore the potentials of graph-based algorithms to identify anomalies in distributed systems. These systems can be heterogeneous or homogeneous, which can result in distinct requirements. One of our objectives is to provide an in-depth look at graph-based approaches to conceptually analyze their capability to handle real-world challenges such as heterogeneity and dynamic structure. This study gives an overview of the State-of-the-Art (SotA) research articles in the field and compare and contrast their characteristics. To facilitate a more comprehensive understanding, we present three systems with varying abstractions as use cases. We examine the specific challenges involved in anomaly detection within such systems. Subsequently, we elucidate the efficacy of graphs in such systems and explicate their advantages. We then delve into the SotA methods and highlight their strength and weaknesses, pointing out the areas for possible improvements and future works.

Index Terms—Anomaly detection, deep learning, distributed systems, dynamic systems, graphs, heterogeneous systems.

I. INTRODUCTION

ANOMALY detection refers to finding abnormal behavior or patterns in the data or a system that does not match the expected behavior [1]. In other words, a non-benign change in the known and correct behavior of a system can be detected as an anomaly. Anomaly detection is critical in both homogeneous and heterogeneous distributed systems. In heterogeneous distributed systems, different components, from small variations like sensors, work all the way up to large components like control facilities to achieve the overall goal of the system. All these components are spread out in a big network; hence the

Manuscript received 19 May 2022; revised 16 April 2023; accepted 1 June 2023. Date of publication 5 June 2023; date of current version 27 November 2023. This work was supported by Siemens Technology. Recommended for acceptance by B. He. (Armin Danesh Pazho and Ghazal Alinezhad Noghre contributed equally to this work.) (Corresponding author: Armin Danesh Pazho.)

Armin Danesh Pazho, Ghazal Alinezhad Noghre, and Hamed Tabkhi are with the University of North Carolina at Charlotte, Charlotte, NC 28223 USA (e-mail: adaneshp@uncc.edu; galinezh@uncc.edu; htabkhiv@uncc.edu).

Arnab A Purkayastha is with the Western New England University, Springfield, MA 01119 USA (e-mail: arnab.purkayastha@wne.edu).

Jagannadh Vempati and Otto Martin are with the Siemens Technology, Princeton, NJ 08540 USA (e-mail: jagannadh.vempati@siemens.com; m.otto@siemens.com).

Digital Object Identifier 10.1109/TKDE.2023.3282898

TABLE I
EXAMPLE OF DISTRIBUTED SYSTEMS WITH SAMPLE SOURCE OF ANOMALIES

Network	Sources of Anomaly
Power Generation and Distribution	Anomalies in hardware, software, and the network
Smart Video Surveillance	Abnormal and dangerous events, hardware or software issues
Smart Cities	Fault in Infrastructure, safety and privacy issues
Smart Grids	Faults, cyber-attacks, natural disturbances
Social Network	Illegal activities, bullying, spams, offensive content
Telecommunication	Malfunctions, hardware issues, intrusion
Company's Internal Network	Hardware and software issues, unwanted access, intrusion
Factories' Production Line	Fault in manufacturing, problem in manufacturing line equipment
Smart Transportation	Technical issues, hardware and software issues, abnormal trajectory changes
Banking System	Money laundering, fraud transactions, intrusion

word distributed, and they are dissimilar with respect to their structure and data production, hence the word heterogeneous. Homogeneous systems can be seen as a special case of heterogeneous systems where all the components are very similar if not identical.

Many distributed systems are operational in critical and/or important fields (e.g., Power Generation and Distribution systems). Thus, their security and correctness are critical. Vulnerabilities can be seen in every aspect of these systems and in different levels of abstraction; from problematic individual components within a system or a network to seeing abnormalities or irregularities where these components connect with each other to accomplish a more advanced task. Table I shows several distributed networks and possible sources of anomalies in them. Anomaly detection in these systems must meet many requirements, making the task even more complicated. Anomalies must be detected by processing large quantities of data, considering the nature of anomalies, which makes detecting them a challenging task and the real-world setup limitations.

In recent years, neural networks have shown great potential and are being explored extensively. They have opened new

avenues and increased interest in extending deep learning approaches for anomaly detection [2], [3]. Traditional methods are mostly dependent on handcrafted features defined by domain experts which makes them less generalizable and flexible for different domains. Feature selection can be time-consuming and error-prone. In contrast, deep learning methods are highly adaptable to the specific requirements of each domain. Deep learning models help with automation as they need minimal effort and supervision for identifying the prominent features without manual feature selection. They are more suitable for learning complex information, their result is more generalizable and less prone to over-fitting, and they are model free. Another advantage of deep learning algorithms in the context of anomaly detection is that they can handle large input data more efficiently and scale up compared to traditional methods. This characteristic makes them ideal for anomaly detection in complex distributed systems. Deep learning-based algorithms have proved to be more effective and generally show better results. This is due to the fact that they are capable of building more enriched features and detecting complex patterns compared to traditional methods.

Real-world distributed systems are an intertwined network of numerous components, each of which representing distinctive properties, changing over time. Graphs prove to be advantageous in capturing the relational dynamic of these components as well as their individual features. Mapping a system to nodes and edges of a graph, allows for a better comprehension of the system. It is noteworthy that not all types of graphs are capable of capturing every aspect of a system. While conventional graph representations are adequate for homogeneous systems, heterogeneous systems require the more complex attributed graphs. Attributed graph is a type of data structure where each node and edge is associated with a set of attributes or properties. In heterogeneous systems, they allow for the representation of multiple variations of entities and relationships, each with their own unique attributes [4]. On the other hand, to address the changing nature of distributed systems, dynamic graphs come to aid. In dynamic graphs, nodes and edges can appear, disappear, or change over time, rendering them ideal for dynamic systems. Hence, When surveying graph-based methods, we analyze their ability to handle attributed and dynamic graphs.

Numerous studies have reviewed techniques and tools for general anomaly detection problems [2], [3], [5], [6], [7]. Some of the existing surveys focus on anomaly detection in Big Data such as [8], [9]. In addition, certain studies investigate methods utilized for anomaly detection in specific narrow domains such as fake news detection [10], social media interactions [11], financial transactions [12], etc. Several articles concentrate on the applications of graphs for identifying anomalies in different systems. [13] overviews anomaly detection algorithms in graphs of dynamic networks; however it mostly mentions the traditional methods and neglects state-of-the-art deep learning methods without mentioning the challenges and difficulties of anomaly detection in such systems. A recent study [14] tries to fill this gap and focuses on deep learning methods for anomaly detection in graphs. [14] also mentions the challenges in this area, but falls short in categorizing the challenges and analyzing the real-world requirements and constraints. Both [13], [14] do not provide

different use cases and analyze them with respect to review algorithms. Another work [15] provides a general overview of methods for anomaly detection in data represented as graphs and discusses the challenges and use cases of these methods. However, this work did not focus on deep learning methods for anomaly detection, as these techniques were not yet widely established at the time of the study.

The major focus of this survey is to provide a comprehensive overview of the state-of-art graph-based techniques to solve the problem of anomaly detection. In particular, we look at real-time complex distributed systems and qualitatively model those to identify and analyze various methods in anomaly detection that utilize the benefits of graphs. In a nutshell, we make the following notable contributions to this survey:

- Modeling three conceptual use cases and employing them for exploring the requirements, challenges, and benefits of anomaly detection algorithms.
- Identifying the logical, algorithmic, and implementation requirements and challenges of anomaly detection in real-world distributed systems.
- Comparing and contrasting the different characteristics of the graph-based anomaly detection algorithms, tools, and techniques in-depth for dealing with anomalies and qualitatively modeling them over the three conceptual use cases that we introduce.
- Providing future research direction for graph-based anomaly detection in distributed systems.

The remainder of this paper is organized as follows: Section II describes three use cases that are the representatives of real-world distributed systems. Section III identifies the challenges and requirements of anomaly detection in the context of real-world distributed systems. Next, Section IV argues the benefits of moving toward using graphs in distributed systems. Section V aptly delve into the graph-based approaches and the means of utilizing graphs for anomaly detection. Finally, Section VI discusses and compares these methods and their complexity, the new challenges and requirements that graphs add, and the future directions that this research field can take.

II. CONCEPTUAL USE-CASES

In this section, we introduce three conceptual use cases that will be used throughout the paper for better understanding. While these three conceptual use cases are good samples of heterogeneous distributed systems, if necessary, these systems can be segmented into homogeneous subgroups to fit the anomaly detection method's requirements.

We have made sure to cover different levels of abstraction to help with the development of a richer understanding of the concepts. The aim is to clarify that anomalies can occur in all levels of a system, whether it is a small component or it is a large subgroup of the whole system.

A. Smart Surveillance System

Here we use an example from [16]. Real-time Edge Video Analytics for Multi-camera Privacy-aware Pedestrian Tracking or REVAMP²T [16] is designed for tracking pedestrians across

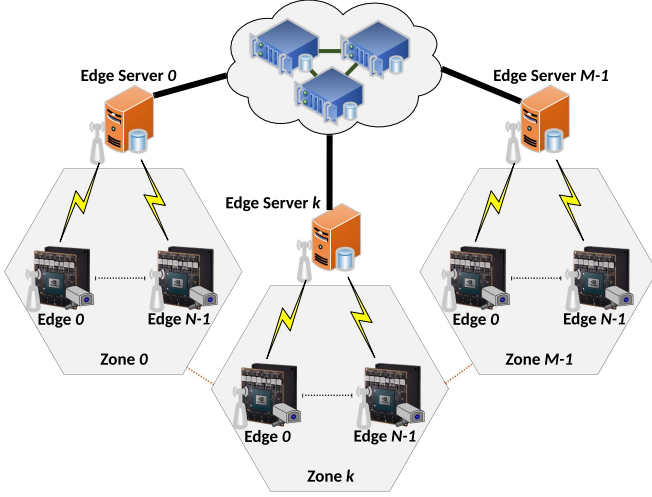


Fig. 1. REVAMP²T [16] setup. This setup can be expanded or shrunk to a new setup. Each edge server only sees the data of its own location and the cloud server is able to see all the locations connected to it. For privacy purposes, a supervised amount of information is transmitted from each location to the cloud server.

multiple cameras. The overview of the model is shown in Fig. 1. The installed edge nodes at each place of interest consist of a number of edge cameras empowered for analyzing streaming video. These cameras are connected to an edge server that acts as a database that contains information about seen objects. The video can be sent to a number of surveillance monitors with the extracted information if necessary and allowed. Also, the whole information can be sent to a cloud server for further processing of less sensitive information. An anomaly can happen throughout this whole complex distributed heterogeneous system. For example, a camera might get broken, or even there can be a hacker trying to steal the extracted information from the edge server.

This distributed system is completely attributed and dynamic, leading to a dynamic attributed graph representation. Benign changes such as removing or updating the components can happen in this system, which should not be counted as anomalies. Other events such as receiving noisy video from a camera, information mismatch between the edge server and the cloud server, and unwanted access to data at any point of the network can be seen as anomalies.

B. Sensor Network

We next look at the conceptual use case of a heterogeneous Sensor Network that illustrates a complex distributed system. To accurately depict the security aspects of such a system, we take inspiration from the Purdue Model for ICS security [17]. The Purdue Enterprise Reference Architecture was created by mapping the interconnections and inter-dependencies of the high-level components of typical industrial control systems (ICS) to provide guidance on how to defend IT (Information Technology) and OT (Operations Technology) systems against malicious actors. Fig. 2 shows the model of such a system

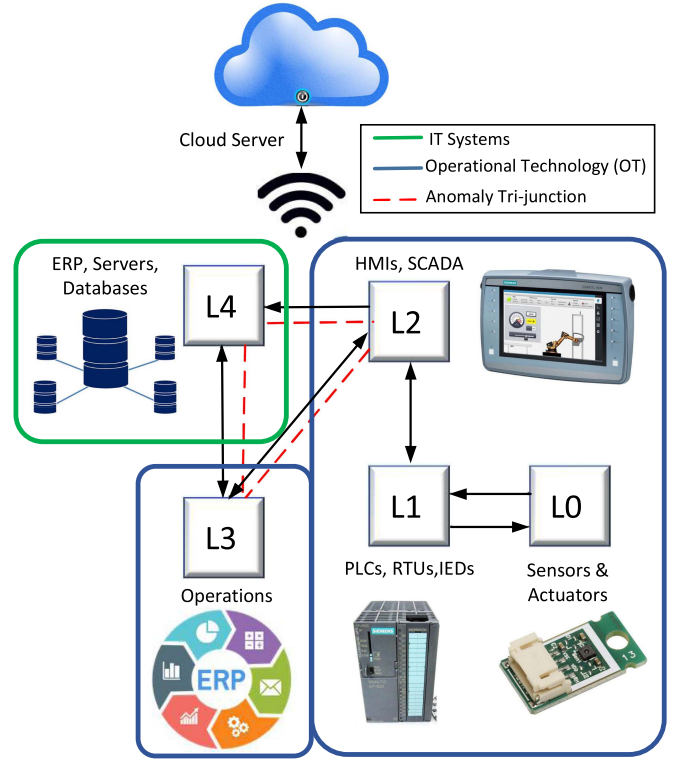


Fig. 2. Sensor network.

modified to incorporate all the possible components of a heterogeneous system. Here, Level 0 (L0) through Level 4 (L4) are the functional levels that span two broad zones, namely IT and OT. OT systems can be further classified into two sub-categories which will be discussed as follows.

- *L0*: Physical components that build products. Eg. motors, pumps, sensors, valves, etc
- *L1*: Systems that monitor and send commands to devices at L0. Eg. PLCs, RTUs, IEDs, etc
- *L2*: Overall process controllers. Eg. HMIs, SCADA, etc involves humans managing and controlling the processes
- *L3*: Management of production workflows. Eg. batch management, operations management, etc
- *L4*: ERP software, databases, email servers and other systems that manage the logistics of the manufacturing operations and provide communications and data storage

The anomaly tri-junction barrier (shown in red) serves as a demarcation of communication between the various levels and possible sources of anomalies.

C. Internal Local Area Network

An internal Local Area Network (LAN) is also a good sample model of a complex distributed heterogeneous system. The graph representation of the system is attributed, dynamic, and directed. It is dynamic because all the components in the network are subject to change. For example, clients can change their devices, the hardware of the server can change (e.g., adding more memory or upgrading the CPU), the mediums for connections

TABLE II
CHALLENGES OF ANOMALY DETECTION

Category	Challenge	Short Description	References
Data and Evaluation	Big Data	Volume, velocity, variety, veracity, and value of big data can be problematic.	[18], [19], [20], [21], [22], [23], [24], [25], [26]
	High-Dimensionality	Many numbers of features for each datapoint can cause sparsity of data which makes anomaly detection more arduous.	[27], [28], [29], [30], [31], [32], [33]
	Heterogeneity	Diversity and variety of data types, formats, and structures in systems.	[34], [35], [36], [37]
	Labeled Data	The number of labeled datasets are alarmingly low for anomaly detection purposes.	[14], [38], [39], [40], [41]
	Unbalanced Data	Anomalies happen rarely, and training on such unbalanced data makes the model biased.	[42], [43], [44], [45]
	Unclean Data	Many proposed models work based on the hypothesis that clean data is available which is not realistic.	[46], [47], [48], [49], [50], [51]
	Metrics and Benchmarks	Lack of Proper Metrics and benchmarks makes the assessment of proposed models difficult and impossible.	[52], [53], [54], [55], [56]
Nature of Anomaly	Changing Nature	Novel and new types of anomalies can happen, and anomalies may adapt themselves with anomaly detection algorithms.	[57], [58]
	Disparate Sources	Anomalies can happen in many different layers and parts of systems.	[59], [60], [61]
	Obscured Anomalies	Outliers can be in disguise, especially the anomalies that are caused by a smart entity.	[14], [62], [63], [64]
	Noise Resilience	In many cases, noises in the input data can mislead the models and be detected as false positives.	[65], [66]
Real-World	Inference Time	In real-world applications, it is vital to detect anomalies in a timely manner to take the appropriate action.	[67], [68], [69], [70], [71]
	Privacy	In many applications, the data of the users should be protected.	[72], [73], [74], [75], [76], [77], [78], [79], [80]
	Ciphered Data	In many environments, the exchanged data is ciphered.	[81], [82], [83], [84], [85]
	Dynamic Systems	Real-world systems often change over the time and have dynamic nature.	[14], [86], [87], [88], [89], [90], [91], [92]
	Interpretability	In real-world setups, the cause of the anomaly is important.	[93], [94], [95]
	Domain Shift	The model trained on one domain, cannot generalize even to a slightly different domain.	[96], [97], [98], [99], [100], [101], [102]

can change, a client can get removed, or the framework of each platform can get updated.

To emphasize, benign changes to the system must not be counted as an anomaly. We are dealing with a dynamic system that does not have a unique normal state. Thus, capturing anomalies is a more difficult task than when we are dealing with static systems.

III. CHALLENGES

When it comes to anomaly detection, especially in real-world systems, a number of requirements must be met. In this section, we identify the general requirements and challenges of anomaly detection in real-world distributed systems. We look at these challenges from four aspects, each of which examines anomaly detection from an alternative standpoint: Data, Anomaly Nature, Graph, and Real-world aspects.

Table II summarizes all the requirements and the challenges that follow them. In the rest of this section, we discuss each one of them in more detail.

A. Data and Evaluation Aspect

1) *Big Data*: When it comes to anomaly detection with learning approaches, the data becomes an immediate challenge. In many applications, the amount of data that has to be processed is immense, pushing the problem into the Big Data category. Big data meaning is best described through the 5Vs of Big Data: Volume, Velocity, Variety, Veracity, and Value, each exhibiting a series of challenges. For anomaly detection, especially in distributed systems, a huge amount of data (Volume) is being generated and processed at a very high speed (Velocity), containing multiple data types and formats coming from several sources (Variety), with uncertainty about the data and its quality

(Veracity), all or most of it important, and must be considered for extracting a worthy output (Value).

2) *High Dimensionality*: This is another challenge that arises from the data. In many applications, the data that should be processed has a large number of features (dimensions). Huge dimensions of the dataset can cause numerous challenges in many applications. However, these challenges are exacerbated in the context of anomaly detection. The phrase “curse of dimensionality” generally refers to the problems that arise when the number of dimensions increases. A dataset is high-dimensional when we can see the curse of dimensionality. The growth of dimensions will increase the size of the data accordingly and causes sparsity which eventually results in the data points having relatively the same distance from each other. As a result, it would be more difficult to detect hidden anomalies in high-dimensional space. This issue is still a controversial topic in scientific society.

3) *Heterogeneity*: Distributed systems, specially in real-world scenarios, normally include various types of entities, each with their own set of characteristics and behaviors. Consequently the relation between these components is also specific based on the type of the component. The data produced from such systems are heterogeneous, and many methods are not able to handle complex variable heterogeneous data. Thus, heterogeneity is another challenge for the task of anomaly detection. Heterogeneous systems often have large and complex data structures, with a high degree of interconnectivity and variability.

4) *Labeled Data*: In data-driven approaches, the quality of the data has a vital role in the outcome of the model. When it comes to anomaly detection, the number of labeled datasets is alarmingly low. This fact encourages the use of unsupervised or semi-supervised approaches. Thanks to new technologies and advancements in devices, a huge amount of data is available. However, in most cases, this data does not incorporate information regarding anomalous behavior. On the other hand, it is both expensive and time-consuming to obtain high-quality labels for available data. In many cases, anomalies must be discovered and labeled by experts who have field-specific knowledge, making the task harder and more demanding. Also, we should consider the noises added to the dataset by imprecise labeling. Undetected anomalies can have a huge cost in critical situations of real-world applications. Thus, using supervised learning is somewhat problematic in the context of anomaly detection. However, several approaches try to overcome this challenge by taking advantage of synthetic labeled datasets, but many other approaches have moved toward using semi-supervised, unsupervised, and self-supervised algorithms.

5) *Unbalanced Data*: Another point to be mentioned is that anomalies are rare. Supervised learning approaches need to see enough examples while training to learn and understand anomalies. However, the number of anomalies is usually very subtle, causing the dataset to become unbalanced. Anomalies are out-of-the-ordinary incidents, and it is not rational to expect enough samples of each type of anomaly to be available in the dataset. This challenge requires the direction of solving the problem to adapt to it. The abundance of normal data points may result in neglecting the detection of anomalous data points which are valuable in anomaly detection. This problem arises

from the fact that many machine learning algorithms are based on the hypothesis that classes of data have the same distributions which is not realistic in the case of anomaly detection.

6) *Unclean Data*: Many approaches, such as anomaly detection with autoencoders (discussed in Section V need datasets that only contain normal datapoints for training. These approaches learn the normal state and decide whether an anomaly has happened or not based on the divergence from features of normal datapoints. However, as discussed, anomalies can be rare and very subtle. There is a very high chance that an anomalous datapoint slips through the fingers of the responsible individual or team, or it might not even be considered an anomaly until later. As a result, often datasets include undesired outliers that make the dataset unclean. This unclean data can confuse the model about the features and representation of the normal datapoints and deteriorate the accuracy of the model.

7) *Metrics and Benchmarks*: On top of all the challenges, the most important one seems to be the fact that there is no unified metric or benchmark, where models can assess themselves and compare the results. Various kinds of metrics have been introduced and we discuss a number of important ones in Section VI-A, but almost none of the works assess those metrics on a single unique domain, to make the comparison between models possible.

B. Anomaly Nature

1) *Changing Nature*: Anomalies have a specific nature and a collection of characteristics that lead to a particular set of challenges. The first and most important challenge is the changing nature of anomalies. Anomalies are not predefined static occurrences. They can transpire in different shapes and formats, and not all of them can be considered prior to the occurrence of an anomaly. There can always be novel anomalies that have never been considered. They can adapt to anomaly detection mechanisms and evolve to pass through the defenses of anomaly detection algorithms. Because of this characteristic, most algorithms tend to skew toward online learning, unsupervised learning, or semi-supervised learning, where there are no predefined anomalies, and the algorithm learns based on what it sees at the moment on its own. In other words, most of the algorithms make an effort to eliminate external artificial biases and proceed toward generic solutions. This instigates the use of online learning, where the algorithm is constantly learning about the behavior of the system.

2) *Disparate Sources*: Distributed systems typically incorporate multiple layers (resources), resulting in a complex and intricate system as a whole. The role of each of these layers, disregarding its majority or minority can not be neglected. Anomalies can penetrate each and every one of those components, making it very difficult to track them. Especially in large facilities, the number of components grows exponentially, and all of them are potential points of anomaly. This disparate source of anomalies is one of the challenging complications that is troubling anomaly detection algorithms.

3) *Obscured Anomalies*: In addition to these challenges, a considerable number of anomalies, predominantly the ones that

came into existence because of a smart entity, do not like to be found. They might be disguised, or hidden. For example, a cyber-attack tries to be as masked as possible. On the other hand, there are anomalies that are not concealed and are just the result of a malfunctioning component in the system. Being conscious of both of these aspects and prepared for obscured anomalies is another challenging task on the way to generic anomaly detection.

4) *Noise Resilience*: Systems are subject to experiencing a profuse number of internal or external noises. These noises are not anomalies until they pass a certain threshold. However, they are still an irregularity in the system. The anomaly detection algorithms must be capable of distinguishing between noises and anomalies. If not, they might raise too many false positive alarms, resulting in supererogatory attention and costs. These noises can have a wide variety of origins. Some of them are back-breaking to discover, and it is best to introduce a means of filtering those noises out inside the algorithm itself, instead of putting an effort to find the source of the noise.

C. Real-World Challenges

1) *Inference Time*: When it comes to using anomaly detection, time is of the essence, especially in real-world applications. It is expected, time-wise, that the detection of the anomaly does not sit far from the actual happening of the anomalous behavior. Particularly, in a critical environment such as healthcare applications, a long latency in detecting a malfunctioning device might lead to catastrophic results, and it is impermissible. Thus, necessary actions must be taken to avoid such a latency. However, anomaly detection techniques based on deep learning are often a heavy task, requiring a colossal amount of computational power.

2) *Privacy*: Privacy is another important hindrance that must be considered in a real-world environment. Anomaly detection algorithms are involved with all the data circulation and the state of the system to the furthest extent. They know everything about a system, and if that kind of information leaks out of the environment, it can lead to cataclysmic undesirable results. Hence, an anomaly detection algorithm must be obliged to preserve privacy and be as secure as possible if it desires to be deployed in a real-world environment.

3) *Ciphered Data*: The fact that anomaly detection algorithms need to be aware of everything that is going on related to a system brings about another challenge. Many systems such as financial systems, social media, or facilities like power generation and distribution networks do not intercommunicate in plain, raw data. They encode the data and then transmit it. This ciphered data usually is not related to the original data in an obvious manner and that is the whole purpose of encoding the data. Hence, algorithms face yet another arduous challenge. For example, wireless communication between a sensor and the server contains an encoder at the edge transmitter, ciphering the data for privacy and security reasons, and a decoder at the receptor that deciphers the data for further processing. When an algorithm tries to discover anomalies on the medium (here the wireless connector) it cannot access the raw data before the transmission stage.

4) *Dynamic Systems*: Furthermore, static systems are extremely rare in a real-world environment. Real-world distributed systems are extremely dynamic. Components get removed, added, changed, or updated. These benign transformations must not be counted as anomalous behavior. The anomaly detection algorithms might consider these transformations as anomalies. If so, the number of false-positive alarms might rise to a point that makes the algorithm non-functional and impractical for real-world adoption. On the other hand, the anomaly detection model should be flexible enough to learn about the updated features and structure of the network to fully incorporate existing information. Online learning is a tool for overcoming this challenge, however, this is still a very hot topic in research communities.

5) *Interpretability*: Another important challenge is the interpretability of the anomalies. Particularly, in real-world deployments, the tendency to see the actual type of anomaly and not just the detection is more desired. One reason behind this is that recognizing the source of the anomaly, realizing the actual type of the anomaly (e.g., cyber attacks on sensor 1 A), and informing the responsible individuals is very important. Finding the root of the anomaly is important for maintaining the availability of the system.

6) *Domain Shift*: Last but not least, anomaly detection approaches become handy, when you are able to generalize over different domains. However, since we are dealing with learning-based approaches, this domain shift comes with a huge cost. Neural Networks learn about the environment and adapt to it, making it difficult to switch to another domain. This fact discourages the use of supervised learning and also pushes novel techniques toward online learning. This challenge is shared between graph-based anomaly detection and other approaches.

IV. MOTIVATION: WHY GRAPH-BASED?

The use of graphs for anomaly detection is a very recent topic of interest in the machine learning and deep learning community. In this section, we highlight the benefits of graphs in the context of anomaly detection.

Ability to Represent Complex Dependencies: Graphs are widely used for modeling and analyzing complex systems with non-Euclidean data and intricate relationships among system components. This characteristic is particularly useful for anomaly detection problems since anomalies in nature can arise from complex interactions between variables that are hard to capture using traditional models.

Flexibility: Complex systems consist of many different components each with its own set of characteristics. Graph-based models are highly flexible and can be adapted to a wide range of structures and data types. Graphs suit well for both structured (symbols, images, grid-based data) and unstructured (knowledge graphs, social network data, distributed systems, citation networks, network traffic) that can be easily represented by regular and/or irregular graph structures.

Scalability: Anomaly detection often deals with high-dimensional data with a large number of features. Due to this fact, it is hard to capture the underlying patterns and distribution

TABLE III
SUMMARY OF REVIEWED APPROACHES AND THEIR CAPABILITIES

Approach	Model		Attributed Graphs	Dynamic Graphs	Learning Adaptability	Scalability
Federated Learning	DIoT [103]		✗	✓	S	
	VFL [104]		✗	✓	U	
Autoencoders	[105]		✗	✗	U	✓
	[106]		✗	✗	U	
	[107]		✗	✗	U	
	Robust Deep Autoencoder (RDA) [108]		✗	✗	U	
	Iterative Training Set Refinement (ITSR) [109]		✗	✗	U	
	[110]		✓	✓	U	
	DONE [111]		✓	✗	U	✓
	AdONE [111]		✓	✗	U	✓
	DeepSphere [112]		✗	✓	U	
Graph Embedding	Shallow Encoders	DeepWalk [113]	✗	✗	U, O	✓
		Node2Vec [114]	✗	✗	SM	✓
		LINE [115]	✗	✗	-	✓
		TADW [116]	✗	✗	-	
		NetWalk [117]	✗	✓	U, O	
	Deep Encoders	Graph Deviation Network (GDN) [94]	✓	✗	U	
		AddGraph [118]	✓	✓	SM	
Graph Transformers	[119]		✗	✓	S	
	TADDY [120]		✗	✓	U	
	GTA [121]		✗	✓	U	
Graph Signal Processing	[122]		✓	✓	U	
Graph Contrastive Learning	GCCAD [123]		✓	✓	U, SF	
	CoLA [86]		✓	✓	SF	✓
	SL-GAD [124]		✓	✓	SF	✓

S: Supervised Learning, SM: Semi-supervised Learning, U: Unsupervised Learning, SF: Self-supervised Learning, O: Online Learning

of data for finding outliers. Graphs are useful for capturing the underlying structure of the input data and its relationships in a more compact and interpretable format. This can help with more efficient processing of the data and helps with scalability.

Robustness: Graphs are capable of incorporating relationships and the holistic structure of the data which is particularly beneficial in anomaly detection. Graph-based algorithms analyze each datapoint by taking into account the context related to them which improves the robustness and helps reduce the number of false negatives. This is due to the fact that often outliers do not fit into the global patterns of the system.

Interpretability and Visualization: Graph-based models can provide more insights into the relationships between nodes of the system, making them more interpretable than many traditional models. This can be particularly useful in understanding the underlying causes of anomalies and identifying potential areas for improvement and vulnerabilities of the system.

V. GRAPH-BASED TOOLS AND TECHNIQUES FOR ANOMALY DETECTION

In this section, we aim to introduce and compare techniques that combined with graphs have a great potential for solving

the problem of anomaly detection in distributed systems. The summary of all reviewed works and their capabilities can be seen in Table III.

Many of the surveyed methods are not able to handle heterogeneity or dynamic behavior that might be present in the systems. Approaches that are capable of managing attributed graphs are better suited for heterogeneous systems. It is noteworthy that the inability of certain methods to handle attributed graphs does not mean they can not be adopted in heterogeneous systems. Two viable strategies exist for their utilization. First, large systems can be subdivided into smaller homogeneous sub-systems. Second, a simplified version of data can be employed to accommodate the limitations of such methods.

A. Federated Learning

In the context of anomaly detection in distributed systems and when dealing with Big Data, while graphs can offer the ability to capture complex dependencies and handle scalability and robustness, Federated Learning (FL) can be a solution for improved privacy, model generalization, and more efficient learning. The combination of the two seems to be perfectly suited

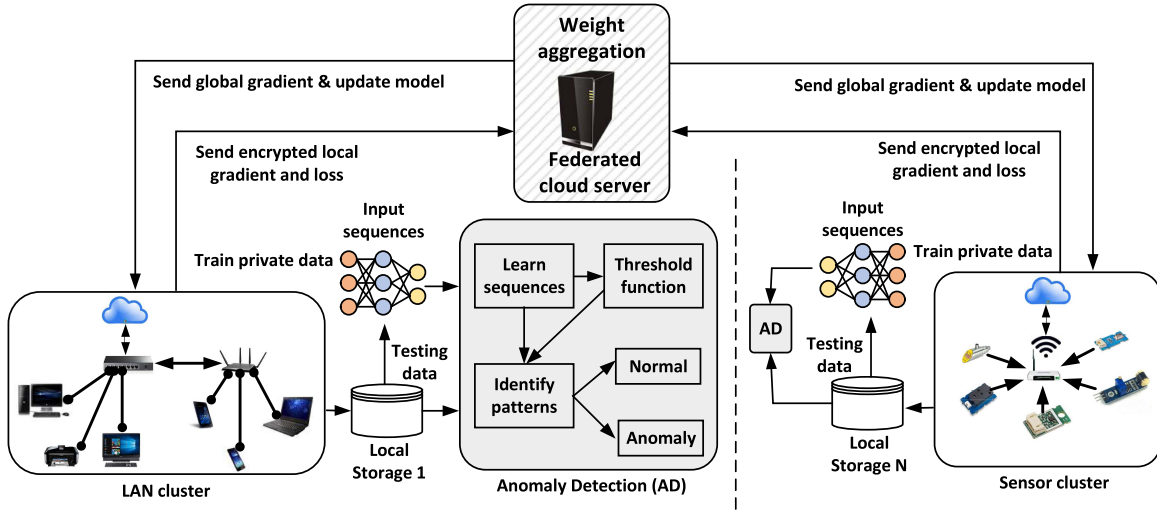


Fig. 3. Federated learning for anomaly detection applied on two distributed clusters viz. LAN and Sensor. Here N is the number of clusters connected to the Federated cloud server. Each network has its own testing and training dataset, local storage, and loss function. The server receives individual weights from all the network's local models and then aggregates them based on their useable graph attributes while sending back the global weights.

for anomaly detection in distributed systems, and it is a great area to be explored by scholars and researchers.

Federated Learning is a technology that has recently emerged as an alternative to centralized systems. It focuses on collaboration while preserving the security aspect of client data information used for training machine learning algorithms [124], [125], [126], [127], [128]. It also greatly reduces communication overhead [129]. FL employs collaborative and experiential learning by training without the need to transfer data over to a centralized location. This feature has recently led to a rise in the applicability of FL in a variety of applications, ranging from medical to IoT, transportation, defense, and mobile apps.

Anomaly detection and prediction of static and dynamic time series data have been a popular topic, especially for IoT-based data [130], [131], [132]. These approaches look at multi-sensor systems as a collection of centralized sensors where abnormal sensor behavior is detected by a central model that runs on the server. Such systems are prone to failures, require longer access times, and are vulnerable to malicious invasion attacks leading to a potential data breach from clients to the server [133]. [134] looks at the data collected by IoT sensors for energy-efficient applications like HVAC in smart buildings and proposes a federated stacked Long Short-time Memory model (LSTM) on time series data generated by IoT sensors for classification and regression tasks. [103], on the other hand, proposes a self-learning distributed system for security monitoring.

While such approaches deal with Autonomy (concerns association and communication requirements) nature of systems, they fall short of the benefits of graph-based relational learning. Federated Graph Learning is a fairly recent topic. [135] introduces Federated Graph Learning (FGL) as a seminal paper discussing the definition and challenges of FGL. It further categorizes FGL into four distinct learning types. Graph edges and nodes in each sub-graph heavily overlap each other and therefore play a pivotal role in extracting the features and adding to contextual and relational learning benefits. A sub-classification of this work

was introduced in [104] as a complete methodology to apply the Vertical Federated Learning (VFL) algorithm for graph convolutional networks. The approach, along with Homomorphic Encryption (HE), was developed to ensure privacy while maintaining accuracy.

A general structure of a complex distributed system can be easily depicted using the Federated Learning framework (Fig. 3). Here we show two different network clusters. In this framework, the models are trained at the device level or client side, where they are brought over to the data sources or devices for training and prediction. The updated values are sent back to the federated cloud server for aggregation. One consolidated model gets transferred back to the devices to enable tracking and redistribution of each model to various devices. During the training phase, the input is reconstructed in the output until reconstruction error is minimized, which calculates the threshold value. This threshold value decides whether the observed patterns are anomalous or not.

Table III summarizes the strong points and weaknesses of two discussed works DIoT [103] and VFL [104]. Both of these works are unable to process the additional information provided by attributed graphs which in real-world scenarios are common. It should be mentioned that for supervised models such as DIoT [103] a crucial requirement is available labeled data that is hard to provide. Based on mentioned shortcomings, although FGL addresses data diversity, data security and real-time continuous learning it is not mature enough yet for real-world applications.

B. Autoencoders

Anomaly Detection is often considered an open-set problem, where it is very unlikely to have complete knowledge of all types of possible anomalies. As a result, researchers have focused on developing semi-supervised or unsupervised powerful neural network models such as Autoencoders. The combination of

autoencoders with graphs has become a popular topic as it helps with overcoming the challenges of anomaly detection, specifically in the context of distributed systems.

Autoencoders are typically made of two main modules; encoders and decoders. The encoder part is responsible for mapping the input space to a bottleneck latent space, and the decoder reconstructs the original input based on the latent representation. In order to find the best possible latent representation, they try to minimize the reconstruction error of the original input. Thus, the network will learn to preserve the most informative parts of the input features in the latent representation. They were traditionally used for dimension reduction prior to feeding the data to the main network, but nowadays, they have vast applications in information retrieval, image processing, and anomaly detection.

In the context of anomaly detection, autoencoders can be used as a tool for calculating anomaly scores. After training the autoencoder with data that does not contain anomalous points, if any outlier data points are fed to the network, it will perform poorly. The reconstruction loss of that data point will be larger compared to normal ones since the network is not familiar with this type of data. As a result, the reconstruction error can be used as a measure of deviation from normal data points. This approach has been used for detecting anomalies in high-performance computing systems [105] and has shown promising results. Convolutional Autoencoders can improve the parameter efficiency and training time since they have shared parameters. Works such as [106], [107] have made use of convolutional autoencoders for anomaly detection and achieved significant improvements. DeepSphere [112] proposed a method for detecting anomalous snapshots in a dynamic network. This work adopts an LSTM autoencoder with an attention mechanism. In the constructed hidden space, DeepSphere [112] learns a spherically shaped boundary around the encoded normal representations. As a result, the encoded representation of an unseen anomalous snapshot of the network will fall outside the hyper-sphere and is detected as an anomaly.

All the aforementioned methods are developed based on the hypothesis that clean data (data that does not contain anomalous points) is available for training, but in many real-world cases, we do not have enough data points that satisfy these constrain. Thus, the reconstruction error for anomalous points will be lower, and the accuracy of finding outliers will degrade. To solve this issue, Robust Deep Autoencoder (RDA) [108] inspired by Robust Principal Component Analysis [136], [137], [138] uses a filter layer that separates anomalous data points of input data. By removing these noisy and anomalous data points, the network will be able to better reconstruct the normal data points. On top of RDA [108], they add an anomaly detection algorithm to test the effectiveness of the proposed method. Iterative Training Set Refinement (ITSR) [109] adapts adversarial autoencoder network [139] architecture to add a prior distribution to constructed latent representation and places anomalies to the regions with lower likelihood. By this means, the model will be robust against noises and anomalies in the training data. DONE [111] is another network proposed for detecting anomalous nodes in attributed graphs. DONE makes use of two parallel autoencoders; one for encoding link structure and another one for attributes of

nodes. These autoencoders are trained to preserve proximity and homophily in the network. The proposed loss function is designed to minimize the contribution of outliers, and by minimizing the loss equation, the anomaly scores for each node are calculated. Finally, the top k nodes are reported as anomalous points. They also propose AdONE [111] that makes use of adversarial learning to be able to construct an outlier-resistant network embedding.

Traditional autoencoder networks and convolutional autoencoders are limited to a fixed input length which makes them unsuitable for detecting anomalies in dynamic networks. Also, they are limited to Euclidean data; thus, they are unable to model complex relationships that may occur in a dynamic heterogeneous network. Graph Neural Networks (GNN) can overcome these issues and generalize the network. They can also incorporate multi-dimensional edge and node attributes. Convolutional graph autoencoders have been used in [110] for detecting anomalies. This model uses both node features and edge features. Also, the decoder module has two separate branches for node reconstruction and edge reconstruction. The final reconstruction loss is the combination of edge reconstruction loss and node reconstruction loss and is used as a measure for finding anomalies.

Fig. 4 shows the workflow for detecting anomalies using autoencoder networks in the LAN Cluster example mentioned in Section II-C. In the case of anomaly detection in a complex system, such as the three conceptual models that we introduced earlier, after training the model with normal data, the graph of the network is fed to the encoder module to be transformed into the latent space. In this case, since we are dealing with a heterogeneous network, the encoder and decoder modules should be able to incorporate the edge and node information and aggregate them to get the most of the available information. Also, at any time, the graph of the network can change. Thus, it is vital to employ a network architecture that can handle dynamic graphs, such as Graph Neural Networks. The decoder will try to reconstruct the original graph of the network from the latent representation. Anomalous edges, nodes, or sub-graphs of the network will have a high reconstruction error. In the most basic approach, only a fixed threshold can be used to decide whether the edge, node, or sub-graph should be considered anomalous or not.

For more clear scrutiny, Table III shows the different features of reviewed works. As mentioned before, one important characteristic of the autoencoder structure is that it is trained in an unsupervised manner that directly solves the problem of the availability of labeled data. Also, the changing nature of anomalies will not be a problem in these types of models since they will learn the normal behavior of the system and anything different from that will be counted as an outlier.

C. Graph Embedding

The first obstacle that we face in many tasks like anomaly detection in large networks is finding a way to map the data hidden in the network graph into a low-dimensional space. To do so, there are many different approaches. In general, we can

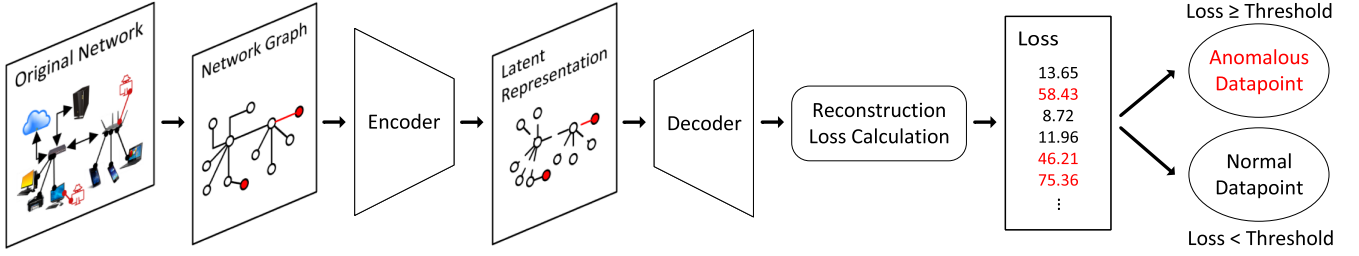


Fig. 4. Graph autoencoder network for anomaly detection in the LAN Cluster example introduced in Section II-C. Two malicious users (equal to anomalous nodes in the network graph) have connected to the network. The network graph is fed to the autoencoder and the reconstruction loss for nodes and edges will be calculated. In this simple case, just by comparing the reconstruction loss to a constant threshold, the anomalies can be detected, but these processes can be more advanced and precise.

separate these methods into two different classes. The first type of encoder is “Shallow Encoders” which will transform each node of a graph into exactly one vector in the latent space. On the other hand, there are methods called “Deep Encoders” which get the use of more complicated networks and are able to generate more complex embedding compared to “Shallow Encoders.” In the following, we will see details of each of these methods and their strengths and weaknesses. After obtaining the latent representations, an anomaly detection algorithm can be used on top of the embedding network in order to detect outliers and anomalies.

1) *Shallow Embedding*: Shallow Embedding methods try to find a unique latent representation for each vertex of a graph. The main difference between different approaches to shallow embedding is in the definition of similarity function. The similarity function basically describes how relationships in the latent space are mapped to the original input space. DeepWalk [113] is a random walk approach for node embedding that tries to find the best embedding that preserves similarity. Each random walk is an unbiased sequence of nodes with a fixed length. Authors claim that random walks can be treated as sentences of a text since the frequency of occurrence of the nodes in random walks follows the power law. DeepWalk [113] tries to find the feature representation for each node such that it maximizes the likelihood of visiting nodes seen in the random walks starting from that particular node. Another method to be mentioned is Node2Vec [114]. This algorithm makes it possible to have more flexible random walks in order to obtain richer latent representations. It combines breath-first sampling and depth-first sampling together to introduce a flexible biased sampling strategy that allows local and global views of the network.

Similar approaches such as LINE [115], and TADW [116] have been proposed. One might think of these algorithms as finding a simple look-up table for assigning each node in the original graph to a latent representation. As a result, these kinds of methods have limitations when we want to apply them to large heterogeneous networks. The first limitation comes from the fact that each node has to have its own unique embedding and there are no shared parameters in these kinds of networks. As a result, when the number of nodes grows, the number of parameters will grow respectively, and we will need $V \times D$ number of parameters where D is the dimension of latent space. Also, they are unable to generalize to unseen nodes, and we

cannot use them in dynamic networks. NetWalk [117] solves the problem with the changing networks and dynamically updates the representations as the network evolves. This model tries to satisfy two constraints: clique constraints which minimize the pairwise distances between representations of vertices in each random walk to preserve locality and autoencoder constrain, which serve as a global constraint and minimize the reconstruction error of input using the output embedding. Still, there are more issues with these types of graph embedding algorithms; another problem is that in complex heterogeneous systems, each node may have its own specific features, but “Shallow Encoders” are unable to make use of these node-specific features.

2) *Deep Embedding*: Most of the methods that we have discussed until now are not capable of modeling more complicated dependencies such as complex inter-sensor relationships, but “Deep Encoders” have more capacity and are able to build richer latent representations. Graph Deviation Network (GDN) [94] introduced a structure learning approach in which the graph edges are initially unknown and have to be learned. GDN [94] is consist of four important components. The first component is Sensor Embedding which captures the characteristics of each node or sensor. In the next step, Graph Structure Learning Learns the complicated relationships between pairs of sensors and models them to the edges of the graph. After the construction of the graph, Graph Attention-Based Forecasting predicts the behavior of sensors in the future time step. Finally, Graph Deviation Scoring will compare these predictions and the actual values in each time step and identify anomaly points that deviate from expected values. AddGraph [118] is able to aggregate more information such as structural, temporal, and content features to be able to build a more powerful embedding for anomaly detection in dynamic graphs. This model makes use of a graph convolutional neural network (GCN) for capturing structural and content features. By adding Gated Recurrent Units (GRU) with attention modules, AddGraph [118] makes it possible to combine the long-term and short-term states of each node. Finally, based on the representation that contains structural, temporal, and content features, AddGraph [118] computes the anomalous score for edges using a single-layer network.

Smart Video Surveillance Systems (Section II-A), Sensor Network (Section II-B), and LAN Network (Section II-C) all contain many nodes. As a result, Shallow embedding techniques are not adequate for modeling these kinds of networks. We

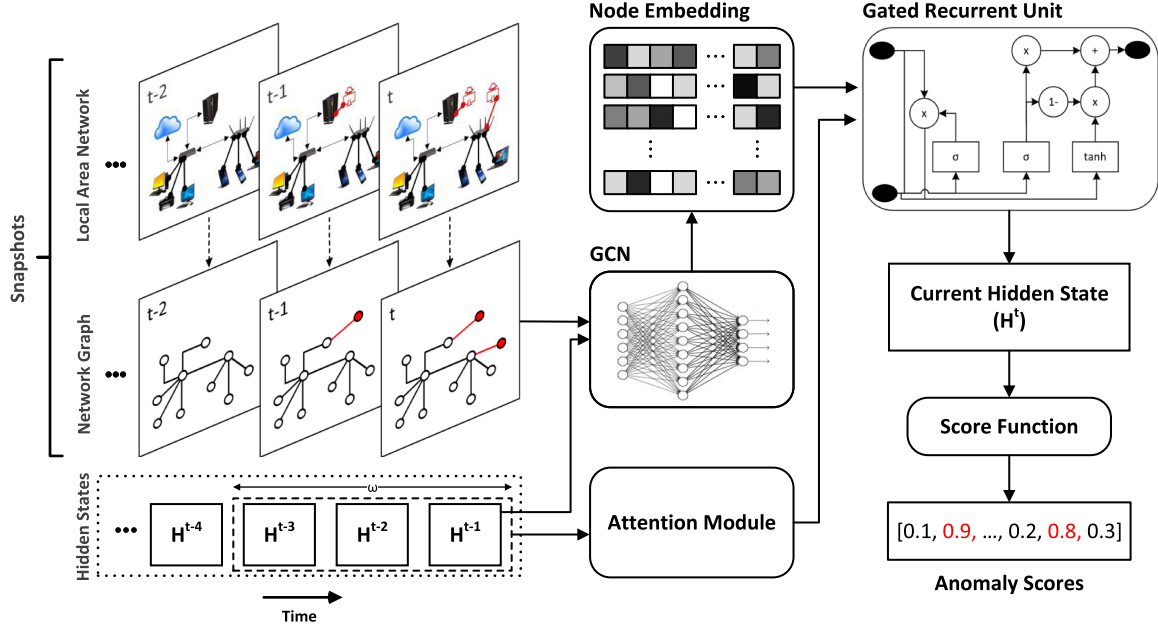


Fig. 5. This figure shows the AddGraph model applied to the LAN Cluster conceptual use case discussed in II-C. In the first step, Graph Convolutional Network (GCN) takes the graph snapshot at time step t and combines it with the hidden state of the previous time step to construct the node embedding. GRU and attention module combine the long-term and short-term states to generate the current hidden state. In the last step, the scoring function assigns an anomaly score to edges based on the nodes connected to them.

should also consider that all of these three conceptual use-cases are also dynamic (as an example adding a new camera to the Smart Video Surveillance System changes the graph of the network); thus, methods like DeepWalk [113], and Node2Vec [114] are not suitable since they are unable to generalize to unseen nodes. Another important issue is that nodes and edges can have features with different schemes that are useful for anomaly detection. Consider the Sensor Network; in each time step, a huge amount of data is generated by a large number of interconnected sensors. The data from each sensor can be related to other sensors with a complex non-linear relationship. Shallow embedding techniques fail to take advantage of this kind of information. Thus, Deep Embedding methods that are more complicated such as GDN [94], and AddGraph [118] can be helpful in this case. Based on what we discussed, let's use one of the introduced examples discussed in Section II for better understanding. In the LAN Cluster example, AddGraph seems to be a suitable choice.

Fig. 5 shows the structure of AddGraph which is applied to the LAN Cluster example. Previous hidden states and current snapshot of the network graph is used for constructing the node embedding. Also, an attention module and GRU combine the long-term and short-term states to generate the current hidden state. In the final step, a scoring function is responsible to assign a normality score to the current hidden state. The main points of discussed techniques are available in Table III.

D. Graph Transformers

The concept of attention mechanism introduced by [140] was first used for natural language processing. This mechanism

aims to do one-step prediction instead of recurrent processing of the data. By this means, the attention mechanism reduces the path length of the computation, which means reducing the information loss and focusing on the most important features for predicting the output. Adapting this technique to graphs can improve the outcome in many applications, such as anomaly detection. Graph transformer networks are able to automatically generate meta-paths by learning and solving the aforementioned problem enabling many applications such as anomaly detection to work on complex heterogeneous systems [141]. In [119], the model takes advantage of relational graph transformers for finding anomalous nodes in a supervised manner. In this work, first, the heterogeneous graph of the network is extracted, and relational graph transformers and a semantic attention network are used for modeling the complex relationships between the nodes and encoding them, and then classifying them for finding anomalous nodes.

Supervised learning is not possible in many applications due to the lack of labeled data. Using an unsupervised setting, the transformer-based Anomaly Detection framework for Dynamic graphs (TADDY) [120] detects anomalous edges in dynamic graphs. TADDY [120] consists of four main modules as shown in Fig. 6. Edge-based substructure sampling captures the spatial-temporal context of each target edge using the graph diffusion method [142], [143]. For each edge, this module constructs a fixed-length importance-aware set of neighboring nodes. Then, TADDY [120] leverages a novel spatial-temporal node encoding for generating node embedding. After acquiring the embedding from the previous step, a transformer network is used as an encoder to capture spatial and temporal features, followed by a pooling module for aggregating the embedding of all nodes

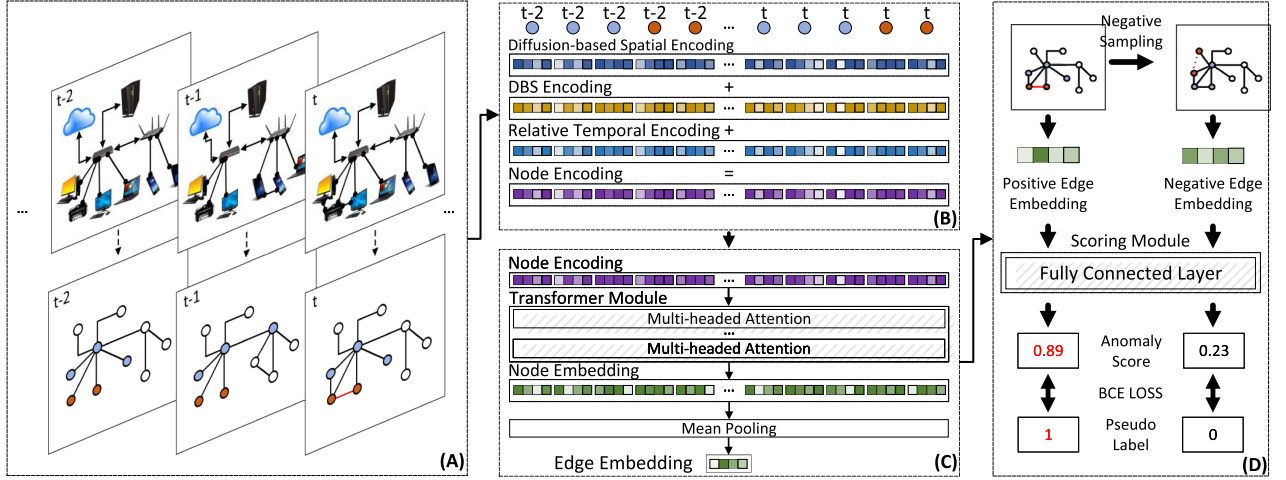


Fig. 6. The figure shows the frame work of TADDY [120]. (A) shows the edge-based substructure sampling module which chooses subgraphs based on the target edge. (B) is the spatial-temporal node encoding module which is responsible for encoding nodes and capturing the global, local, and temporal information hidden in the subgraphs using diffusion-based spatial encoding, distance-based spatial encoding, and relative temporal encoding respectively. The final node encoding is built by combining all previously mentioned encoding. (C) shows the dynamic graph transformer module. This module constructs the edge embedding using a modified multi-headed attention network. Finally, in (D) you can see the discriminative anomaly detector which is trained using samples and pseudo labels generated by a negative sampling strategy. Given that TADDY is only capable of handling non-attributed graphs, we assume that a simplified version of data is utilized to accommodate this limitation.

in the same neighboring set. In the final step, the discriminative anomaly detector (which consists of a fully connected layer).

Graph Learning with Transformer for Anomaly detection (GTA) [84], leverages a modified transformer network. This model detects anomalies on multivariate time series generated from different sensors. These sensors can be related to each other in complicated unknown connections. GTA [84] first learns the dependencies using Gumbel-Softmax Sampling [144] strategy. Once this topological structure is established, a graph convolution block updates each node representation by aggregating neighbors' information and message passing to enrich the representations. Now, dilated convolution [121] is used for extracting temporal context, but with one novel modification. They have used a hierarchical scheme to make GTA [84] capable of capturing temporal patterns with different lengths. On top of these, a more sophisticated and efficient version of transformers is introduced. Multi-branch attention module that is used in GTA [121] extracts long-distance temporal dependencies and neighboring nodes' information. In the anomaly scoring module, the original input time series are divided into training sequences (for the encoder) and label sequences (for the decoder). The decoder predicts the behavior of the time series in the target section, and by comparing the predicted output and the actual values, outliers can be detected.

Capabilities of all mentioned models can be seen in Table III. Unfortunately, none of the anomaly detection models using graph transformers are compatible with attributed graphs which makes them less practical in real applications. Future research can move in the direction of solving this issue to overcome the complex nature of heterogeneous distributed systems.

E. Graph Signal Processing

Graph Signal Processing (GSP) is another technique that utilizes classical signal processing tools like Fourier transform,

filtering, frequency response etc., to process data defined on both regular and irregular graph networks. [145] presents the recent advances in the currently developing GSP tools. Graph signals can be filtered and sampled to apply low-level processing techniques such as denoising and compression. One of the major differences between GSP and traditional Machine learning algorithms is that ML typically considers a graph as a discrete version of a complex network. However, this assumption falls flat for many real-world applications associated with graphs. GSP, on the other hand, looks at existing problems from different perspectives. As an example, defining a graph for sensor-based networks involves choosing edge weights as a decreasing function of the distance between nodes represented by sensors. Observations from similar nodes can lead to a smooth graph function that can detect outliers or abnormal values through high pass filtering or thresholding [146]. Moreover, a sparse set of sensor readings can also be used to build signal reconstruction methods that can be used to save on resources in sensor networks [147], [148].

Anomaly detection with GSP filtering on Wireless Sensor Networks (WSNs) has been a topic of interest in several works. [122] captures proximity information such as data between sensors to capture local anomalous behavior. They present three graph designs and use GSP filtering to find the cut-off frequency and Lambda for the filters. This is used to separate normal and anomalous sub-spaces for unsupervised detection. The anomalous space projections are finally utilized to generate anomaly scores as shown in Fig. 7. The main advantage of such a system is that in addition to raw sensor data, relational characteristics between the nodes, like proximity information between sensors and their environment, can be effectively captured. Graph-based filtering is found to be particularly useful for both regular and irregular graph structures for unsupervised anomaly detection. In Table III, specification of the reviewed model is available. The ability to work on dynamic attributed

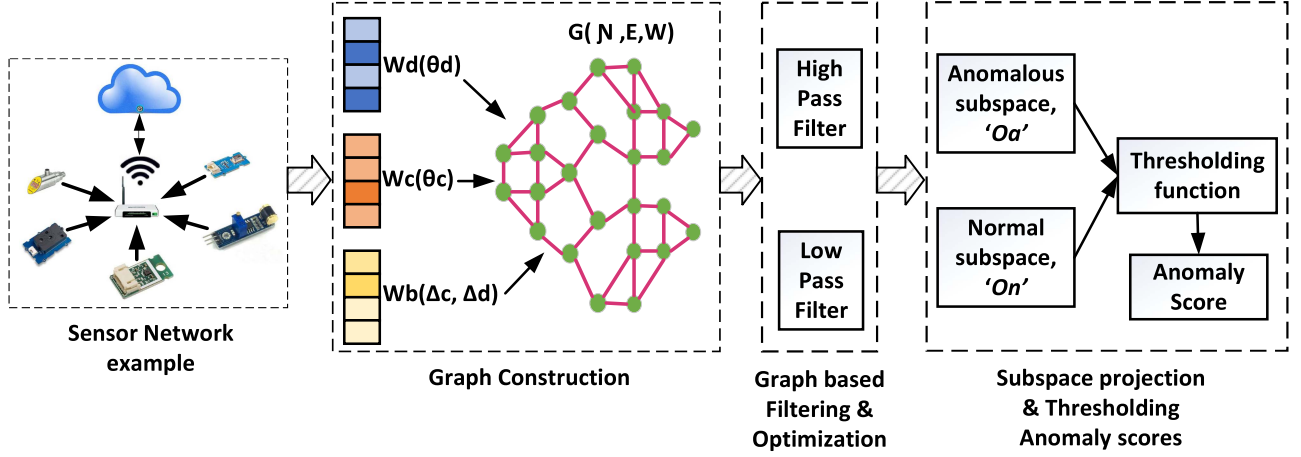


Fig. 7. A small part of the Sensor Network introduced in Section II-B is used to show Graph Signal Processing for anomaly detection. A 3-stage process that begins with Graph construction from 3 adjacency matrices to generate an undirected, weighted, and connected graph $G(N, E, W)$. Filtering and Optimization methods are next used to get the cut-off frequency of the GSP filter that is further projected as a normal and an anomalous subspace. Both these values in addition to a thresholding function are used to generate the anomaly scores.

graphs makes [122] aligned with real-world requirements. On the other hand, leveraging GSP techniques can add deeper perspectives to anomaly detection problems and equip them with more relational information helpful for detecting outliers which is essential in real-world applications.

F. Graph Contrastive Learning

In graph contrastive learning techniques, the objective is to construct a representation by contrasting pairs of data points. The loss function is designed in such a way that by optimizing it, the positive pairs (matching pairs) of data points are brought together, and the negative pairs will be separated. As a result, using this approach, the model is able to learn higher-level representations that are more powerful and distinguishable. In the context of anomaly detection, contrastive learning can be helpful since it is designed to provide a measure of similarity between data pairs. The main idea in GCCAD [123] is to detect anomalies using their distance from average normal points or global context. The key concept is that the outliers will have different features than most points; thus, by contrasting each node with the global context, we can define a measure for detecting anomalies.

GCCAD [123] has shown that the node embedding made by traditional GNN encoders is not able to discern anomalous nodes properly. However, GCCAD [123] is able to construct a more powerful embedding that highlights the differences between anomalous nodes and the global context. This work uses a context-aware loss function in a supervised manner. The loss function is optimized in the GNN encoder which consists of three modules; edge update, node update, and graph update module. The edge update module is responsible for calculating the likelihood of being a suspicious edge (an edge that connects a normal node to an anomalous one) and removing it, and updating the adjacency matrix. These modules are designed to conserve the homophily assumption between neighboring nodes. This assumption suggests that the neighboring nodes

have the same labels. Still, homophily is violated in the case of a connection between a normal node and an anomalous one, and this violation has not been considered in traditional GNN networks. In the next step, the node embedding is updated using message passings by the node update module. Lastly, the graph update module updates the global context. The updated global context is the weighted aggregation of all the nodes. The authors also introduced a self-supervised version of this network named GCCAD-pre [123].

CoLA [86] focuses on anomaly detection in large-scale attributed graphs, which is applicable to real-world problems. This novel network has three main components: pair sampling, a GNN-based contrastive learning model, and the score computation module. The instance pair sampling is designed to generate the pairs of data for the training phase. In contrast with GCCAD [123] that contrasts nodes with the global context, the definition of pairs in CoLA [86] focuses on the relationship between a node and its neighbors. Hence the strategy for defining the pairs is "target node versus local subgraph". The GNN-based contrastive learning module consists of three sub-modules: (1) GNN model, (2) readout module, and (3) discriminator module. After acquiring the pairs, the GNN module will extract embedding for the target node and the local subgraph. Then the readout module uses the average pooling function to create a vector of embedding of all nodes in the embedded subgraph. In the discriminator module, the positive and negative pairs are contrasted, and a score is generated. Finally, the score computation module measures the anomaly score for all nodes, and anomalous nodes can be picked by choosing the nodes with the highest scores. SL-GAD [64], introduced a novel generative and contrastive self-supervised model.

SL-GAD [64] is made of three major components similar to CoLA [86]; graph view sampling, contrastive self-supervised learning, and graph anomaly scoring. In the graph view sampling, for each target node, two local subgraphs are extracted. Next, in the contrastive self-supervised learning module, a GNN constructs the latent representation of the samples. In addition to

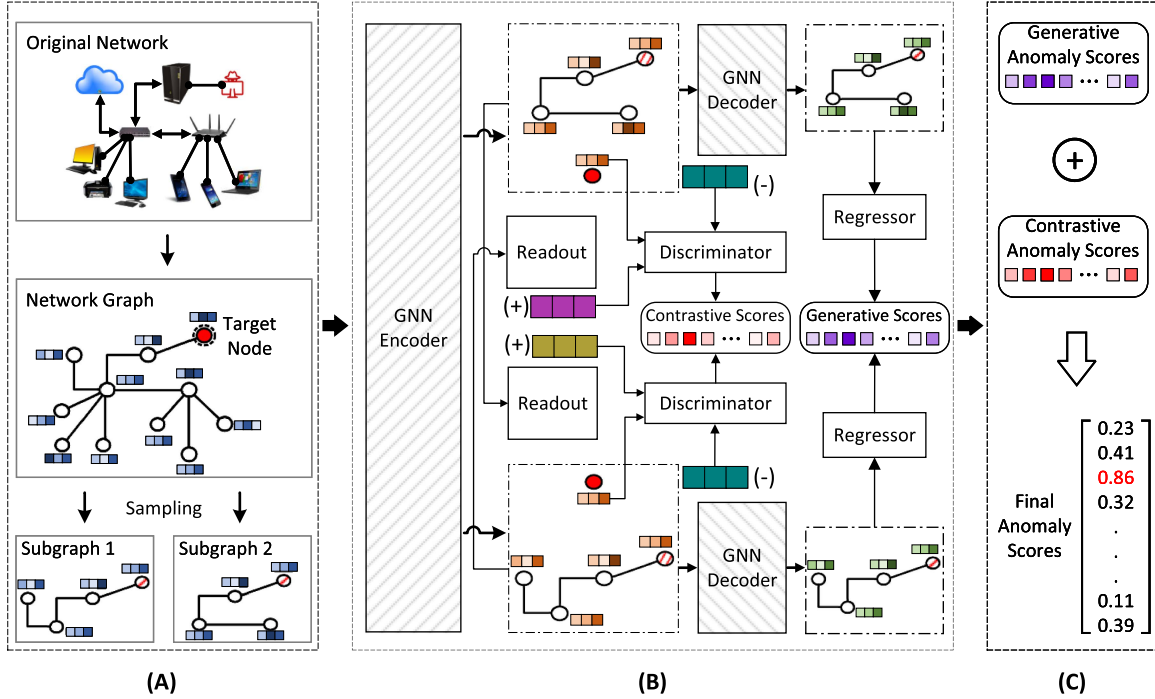


Fig. 8. In this figure we have applied the SL-GAD [64] frame work to the LAN conceptual use-case discussed in Section II-C. (A) shows the Graph view sampling module which chooses the target node and samples two subgraphs. (B) is the generative and contrastive discrimination modules. First, the target node and the two subgraphs are fed to the GNN encoder and graph embedding is created. In the next step, two different objectives, the discriminator and regressor try to capture anomalies in the graph structure and attributes. The generative regression module is designed for capturing anomalies in the attributes of each node, while the discriminator module is responsible for finding anomalies in the structure of the graph. Finally, in (C) the contrastive scores and the generative scores are combined to calculate the final anomaly score.

contrastive scores, SL-GAD [64] also leverages a graph autoencoder network for reconstructing the feature vector of the target node in order to fully utilize the contextual information of target nodes. Finally, graph anomaly scoring predicts the final scores based on both generative scores (from the graph autoencoder) and contrastive scores (from contrastive learning).

Fig. 8 shows the work flow of the SL-GAD [64] applied to the LAN Cluster example introduced in Section II-C. The sampled subgraphs are fed to the GNN Encoder and the encoded representations are fed to two different branches, the discriminative and the generative modules. at the final step, anomaly scores from these two branches are combined to construct the final anomaly score.

Characteristics of all mentioned models can be seen in Table III. As discussed, Graph Contrastive Learning approaches show promising results and are extremely adaptable to real-world scenarios. The discussed works are all able to overcome the dynamic complex nature of distributed heterogeneous systems and the unsupervised/self-supervised learning adaptability of these models can alleviate the problem of available labeled data mentioned in Section III.

VI. COMPARISON AND DISCUSSION

In this section, first, we discuss the metrics that are available for evaluation and then we try to compare the methods discussed in Section V from an algorithmic perspective. Then we discuss

the requirements and challenges that utilizing graphs adds to previously mentioned requirements discussed in Section III, and finally, we discuss the future directions that research in this field can take.

A. Metrics and Comparison

The performance and viability of any anomaly detection algorithm, graph-based or non-graph-based, can be evaluated by Precision, Recall, Accuracy, Receiver Operating Characteristic (ROC) curve, and Area Under the ROC Curve (AUC). While Precision, Recall, and Accuracy are a measure of the true positives, false positives, true negatives, and false negatives, AUC, on the other hand, summarizes the information contained in the ROC curve. Larger AUC values indicate better performance at distinguishing between anomalous and normal observations.

A major challenge while reporting the above-mentioned metrics is the absence of unified evaluation criteria amongst the diversity of algorithms, the application type, and the heterogeneous nature of data and devices involved. As an example, if we want to compare the latency of the two algorithms, we have to make sure that they are both tested on particular hardware, they are getting tested on a specific job, and we have to eliminate different variables that have an effect on that metric.

In contrast, *Time Complexity* is The only metric that can give us some insight into the algorithm's performance. Table IV shows the time complexity of reviewed algorithms. In order

TABLE IV
TIME COMPLEXITY OF REVIEWED ALGORITHMS. FOR SYMBOL DESCRIPTIONS PLEASE REFER TO TABLE V

Approach	Model		Time Complexity	Description
Federated Learning	DLoT [103]		-	-
	VFL [104]		$\mathcal{O}(nm^2)$	-
Auto Encoders	[105]		-	-
	[106]		-	-
	[107]		-	-
	RDA [108]		-	-
	ITSR [109]		-	-
	[110]		-	-
	DONE [111]		$\mathcal{O}(Nd)$	-
	AdOne [111]		$\mathcal{O}(Nd)$	-
	DeepSphere [112]		-	-
Graph Embedding	Shallow Encoders	Deep Walk [113]	-	-
		Node2Vec [114]	$\mathcal{O}(\frac{l}{s(l-s)})$	Per sample complexity
		LINE [115]	$\mathcal{O}(m\hat{n}\hat{d})$	-
		TADW [116]	-	-
		NetWalk [117]	$\mathcal{O}(n \Omega)$	The complexity of walk generation
			$\mathcal{O}(md)$	The complexity of edge encoding
			$\mathcal{O}(cd)$	The complexity of anomaly detection on incoming data points
	Deep Encoders	GDN [94]	-	-
		AddGraph [118]	-	-
Graph Transformers	[119]		-	-
	TADDY [120]		$\mathcal{O}(\tau kmI + T\tilde{n}^2)$	-
	GTA [121]		$\mathcal{O}(4\tau D^2 + 2\tau^2 D)$	This is for the simplest attention module. More complex ones are available in the original paper.
Graph Signal Processing	[122]		-	-
Graph Contrastive Learning	GCCAD [123]		$\mathcal{O}(m)$	-
	CoLA [86]		$\mathcal{O}(kn\hat{R}(\eta + c))$	-
	SL-GAD [124]		$\mathcal{O}(Rnk(\eta + K))$	-

to assess how well an algorithm performs compared to other algorithms, and also with respect to points in Section III, we can refer to Time Complexity. It gives us a good understanding of how fast and efficient the method is, a general idea of the algorithm's complexity, and whether it is suitable for specific tasks or hardware based on the power it needs.

B. Graph-Specific Challenges

Graphs are extremely powerful for catching relational information, especially in distributed systems. They help with addressing challenges and meeting the requirements discussed in Section III, as well as capturing important features of the data like relational features and information. However, there is a cost to all these benefits. While utilizing graphs these challenges should be addressed:

Types of Graphs: Each system can be represented by a specific type of graph. Each type of graph has its features and is suitable

for a group of domains. For example, attributed graphs add more complexity by introducing the features of the connection between the nodes, and the features of the nodes themselves. Also, a graph representing a system might be dynamic or static, depending on the nature of the original system. All these subclasses of graphs add to the complexity of graph-based learning techniques. Subsequently, the introduced models should be able to handle these diverse types of network graphs, but as we discussed in previous sections, many of the models can just work on plain static graphs which is not satisfactory in many applications. [149], [150], [151], [152], [153], [154]

Graph Anomalies: By introducing graphs, we get a better representation of a system. Graphs and Graph Neural Networks make us capable of modeling complicated dependencies in real networks. But if we look at it from another perspective, this adaptation of graph representation introduces different places for anomalies to take place. In graphs, outliers can occur in nodes, edges, subgraphs, and even the whole graph. But models

TABLE V
TABLE IV SYMBOL DESCRIPTION

Parameter	Description
n	number of nodes in the graph
$\tilde{n}$	average number of nodes in a timestamp
m	number of edges in the graph
k	number of nodes in a subgraphs
T	number of timestamps
τ	size of the time window
$\hat{n}$	number of negative samples
l	length of a walk
$ \Omega $	number of walks
s	number of samples
D	number of input dimension
d	latent dimension of node representation
$\hat{d}$	degree of a node
c	number of clusters
R	number of evaluation rounds
I	number of iterations
$\hat{R}$	number of sampling rounds for each node

are usually only able to detect one kind of foresaid anomalies, and there is no unique framework to address them all together. This means that the approaches must specifically determine what is the objective of their detection, which types they handle, and where they show the best performance. [91], [155], [156]

C. Future Research Directions

The most important research direction at the moment seems to be finding a means of quantifying the results of anomaly detection algorithms in a unified manner, thereby enabling comparison in various domains and fields. A great share of all the advancements in technology originates from quantifying and comparing new findings with already existing works. However, considering the variety in this field, comparing algorithms to each other is nearly impossible. First, the best metric for evaluating a model must be identified. Alongside that, an evaluation mechanism, general enough to be applicable to disparate anomaly detection algorithms is also needed. As an example, in computer vision, COCO [157] provides such an environment for emerging algorithms. A framework of testing can also be another fruitful tool. In this direction, creating a simulator, or a system that can serve as a toy sample of a large distributed system can be beneficial as well. To recapitulate, there is a need for a unified evaluation of graph-based and non-graph-based anomaly detection algorithms and techniques. Following this path, an extension that this survey can benefit from is to include experimental results, by conducting a unified benchmarking. This aspect is particularly promising, as incorporating experimental analysis alongside algorithmic and conceptual analysis can yield a more comprehensive and nuanced understanding of the subject matter.

As discussed in the Challenges Section, the second-largest issue in the way of anomaly detection is the lack of a suitable

dataset. Generating or preparing a proper dataset can be a substantial contribution to the field of anomaly detection. However, in the context of anomaly detection, usually, an expert must supervise the process of labeling and collecting data and this fact makes the generated dataset prone to human errors. All of these are added to the actual cost of creating such a dataset. Synthetic datasets are another option, and creating them can be explored through a whole line of research.

Numerous methods of anomaly detection are designed to discover whether an anomaly has happened or not. They are able to notify a responsible person or group to look for further investigation. However, they are not able to identify what sort of anomaly has occurred. A major subject for further work and research is the interpretability of anomalies. Down the road, there is a tendency to distinguish between anomalies using the algorithms at hand. For example, in the Sensor Network mentioned in II-B we like the model to be able to distinguish between a cyber attack and power shortage and raise the correct alarm. If we go further than that, the ways that the model can restore the system or stop the anomaly can be investigated.

VII. CONCLUSION

In this survey, we have discussed several state-of-the-art graph-based approaches for anomaly detection in details. We introduce three unique conceptual use-cases based on real-time complex distributed systems. Our work builds on the existing traditional machine learning and deep learning paradigms used for anomaly detection and goes on to introduce four separate categories of graph neural network algorithms. We further apply our formulated models to each of the methods and provide a thorough review and summarization of the categories. At last, we comprehensively look at graph specific big-data challenges and provide a couple of future research directions in this field.

REFERENCES

- [1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [2] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [3] G. Pang, L. Cao, and C. Aggarwal, "Deep learning for anomaly detection: Challenges, methods, and opportunities," in *Proc. 14th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA: Association for Computing Machinery, 2021, pp. 1127–1130, doi: [10.1145/3437963.3441659](https://doi.org/10.1145/3437963.3441659).
- [4] Y. Wang, Y. Li, J. Fan, C. Ye, and M. Chai, "A survey of typical attributed graph queries," *World Wide Web*, vol. 24, pp. 297–346, 2021.
- [5] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," 2019, *arXiv: 1901.03407*.
- [6] L. Ruff et al., "A unifying review of deep and shallow anomaly detection," in *Proc. IEEE*, vol. 109, no. 5, pp. 756–795, May 2021.
- [7] G. Fernandes, J. J. Rodrigues, L. F. Carvalho, J. F. Al-Muhtadi, and M. L. Proença, "A comprehensive survey on network anomaly detection," *Telecommunication Syst.*, vol. 70, no. 3, pp. 447–489, 2019.
- [8] R. A. A. Habeeb, F. Nasaruddin, A. Gani, I. A. T. Hashem, E. Ahmed, and M. Imran, "Real-time Big Data processing for anomaly detection: A survey," *Int. J. Inf. Manage.*, vol. 45, pp. 289–307, 2019.
- [9] S. Thudumu, P. Branch, J. Jin, and J. J. Singh, "A comprehensive survey of anomaly detection techniques for high dimensional Big Data," *J. Big Data*, vol. 7, no. 1, pp. 1–30, 2020.
- [10] S. Ahmed, K. Hinkelmann, and F. Corradini, "Combining machine learning with knowledge engineering to detect fake news in social networks-a survey," 2022, *arXiv:2201.08032*.

- [11] R. Yu, H. Qiu, Z. Wen, C. Lin, and Y. Liu, "A survey on social media anomaly detection," *SIGKDD Explor. Newsl.*, vol. 18, no. 1, pp. 1–14, Aug. 2016, doi: [10.1145/2980765.2980767](https://doi.org/10.1145/2980765.2980767).
- [12] M. Ahmed, A. N. Mahmood, and M. R. Islam, "A survey of anomaly detection techniques in financial domain," *Future Gener. Comput. Syst.*, vol. 55, pp. 278–288, 2016.
- [13] S. Ranshous, S. Shen, D. Koutra, S. Harenberg, C. Faloutsos, and N. F. Samatova, "Anomaly detection in dynamic networks: A survey," *Wiley Interdiscipl. Reviews: Comput. Statist.*, vol. 7, no. 3, pp. 223–247, 2015.
- [14] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, early access, Oct. 8, 2021, doi: [10.1109/TKDE.2021.3118815](https://doi.org/10.1109/TKDE.2021.3118815).
- [15] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data mining Knowl. Discov.*, vol. 29, no. 3, pp. 626–688, 2015.
- [16] C. Neff, M. Mendieta, S. Mohan, M. Baharani, S. Rogers, and H. Tabkhi, "REVAMP2T: Real-time edge video analytics for multicamera privacy-aware pedestrian tracking," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2591–2602, Apr. 2020.
- [17] T. J. Williams, "The purdue enterprise reference architecture," *Comput. Ind.*, vol. 24, no. 2–3, pp. 141–158, Sep. 1994, doi: [10.1016/0166-3615\(94\)90017-5](https://doi.org/10.1016/0166-3615(94)90017-5).
- [18] I.-Y. Song and Y. Zhu, "Big data and data science: What should we teach?," *Expert Syst.*, vol. 33, no. 4, pp. 364–373, 2016.
- [19] A. Katal, M. Wazid, and R. H. Goudar, "Big data: Issues, challenges, tools and good practices," in *Proc. IEEE 6th Int. Conf. Contemporary Comput.*, 2013, pp. 404–409.
- [20] K. Shin, B. Hooi, J. Kim, and C. Faloutsos, "D-cube: Dense-block detection in terabyte-scale tensors," in *Proc. 10th ACM Int. Conf. Web Search Data Mining*, New York, NY, USA: Association for Computing Machinery, 2017, pp. 681–689, doi: [10.1145/3018661.3018676](https://doi.org/10.1145/3018661.3018676).
- [21] E. M. Knorr and R. T. Ng, "Algorithms for mining distance-based outliers in large datasets," in *Proc. 24 rd Int. Conf. Very Large Data Bases*, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998, pp. 392–403.
- [22] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, New York, NY, USA: Association for Computing Machinery, 2000, pp. 427–438, doi: [10.1145/342009.335437](https://doi.org/10.1145/342009.335437).
- [23] F. Angiulli and F. Fassetti, "Very efficient mining of distance-based outliers," in *Proc. 16th ACM Conf. Inf. Knowl. Manage.*, New York, NY, USA: Association for Computing Machinery, 2007, pp. 791–800, doi: [10.1145/1321440.1321550](https://doi.org/10.1145/1321440.1321550).
- [24] A. Arming, R. Agrawal, and P. Raghavan, "A linear method for deviation detection in large databases," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, AAAI Press, 1996, pp. 164–169.
- [25] K. W. Pettis, T. A. Bailey, A. K. Jain, and R. C. Dubes, "An intrinsic dimensionality estimator from near-neighbor information," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-1, no. 1, pp. 25–37, Jan. 1979.
- [26] N. Tatbul, "Streaming data integration: Challenges and opportunities," in *Proc. IEEE 26th Int. Conf. Data Eng. Workshops*, 2010, pp. 155–158.
- [27] R. Bellman, *Dynamic Programming*. Chelmsford, MA, USA: Courier Corporation, 2013.
- [28] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is "nearest Neighbor" Meaningful?," in *Database Theory — ICDT'99*, C. Beeri and P. Buneman Eds., Berlin, Germany: Springer, 1999, pp. 217–235.
- [29] P. Hall, J. Marron, and A. Neeman, "Geometric representation of high dimension, low sample size data," *J. Roy. Statist. Soc. Ser. B*, vol. 67, pp. 427–444, Feb. 2005.
- [30] J. Ahn, J. S. Marron, K. M. Muller, and Y.-Y. Chi, "The high-dimension, low-sample-size geometric representation holds under mild conditions," *Biometrika*, vol. 94, no. 3, pp. 760–766, 2007. [Online]. Available: <http://www.jstor.org/stable/20441411>
- [31] M. Radovanović, A. Nanopoulos, and M. Ivanović, "Reverse nearest neighbors in unsupervised distance-based outlier detection," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 5, pp. 1369–1382, May 2015.
- [32] S. Sadik and L. Gruenwald, "Research issues in outlier detection for data streams," *SIGKDD Explor. Newsl.*, vol. 15, no. 1, pp. 33–40, Mar. 2014, doi: [10.1145/2594473.2594479](https://doi.org/10.1145/2594473.2594479).
- [33] F. Keller, E. Muller, and K. Bohm, "HiCS: High contrast subspaces for density-based outlier ranking," in *Proc. IEEE 28th Int. Conf. Data Eng.*, 2012, pp. 1037–1048.
- [34] M. Iturbe, I. Garitano, U. Zurutuza, and R. Uribeetxeberria, "Towards large-scale, heterogeneous anomaly detection systems in industrial networks: A survey of current trends," *Secur. Commun. Netw.*, vol. 2017, 2017.
- [35] L. Erhan et al., "Smart anomaly detection in sensor systems: A multi-perspective review," *Inf. Fusion*, vol. 67, pp. 64–79, 2021.
- [36] D. Stiawan, M. Y. Idris, R. F. Malik, S. Nurmaini, and R. Budiarto, "Anomaly detection and monitoring in Internet of Things communication," in *Proc. IEEE 8th Int. Conf. Inf. Technol. Elect. Eng.*, 2016, pp. 1–4.
- [37] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, and M. R. Lyu, "Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention," 2023, *arXiv:2302.06914*.
- [38] N. Görnitz, M. Kloft, K. Rieck, and U. Brefeld, "Toward supervised anomaly detection," *J. Artif. Intell. Res.*, vol. 46, pp. 235–262, 2013.
- [39] S. Suthaharan, M. Alzahrani, S. Rajasegarar, C. Leckie, and M. Palaniswami, "Labelled data collection for anomaly detection in wireless sensor networks," in *Proc. IEEE 6th Int. Conf. Intell. Sensors Sensor Netw. Inf. Process.*, 2010, pp. 269–274.
- [40] M. Luo, K. Wang, Z. Cai, A. Liu, Y. Li, and C. F. Cheang, "Using imbalanced triangle synthetic data for machine learning anomaly detection," *Comput. Mater. Continua*, vol. 58, no. 1, pp. 15–26, 2019.
- [41] W. Lin, J. Gao, Q. Wang, and X. Li, "Learning to detect anomaly events in crowd scenes from synthetic data," *Neurocomputing*, vol. 436, pp. 248–259, 2021.
- [42] M. Frasca, A. Bertoni, M. Re, and G. Valentini, "A neural network algorithm for semi-supervised node label learning from unbalanced data," *Neural Netw.*, vol. 43, pp. 84–98, 2013.
- [43] S. N. Kalid, K.-H. Ng, G.-K. Tong, and K.-C. Khor, "A multiple classifiers system for anomaly detection in credit card data with unbalanced and overlapped classes," *IEEE Access*, vol. 8, pp. 28 210–28 221, 2020.
- [44] H. H. Pajouh, G. Dastghaibfard, and S. Hashemi, "Two-tier network anomaly detection model: A machine learning approach," *J. Intell. Inf. Syst.*, vol. 48, no. 1, pp. 61–74, 2017.
- [45] S. El Hajjami, J. Malki, M. Berrada, and B. Fourka, "Machine learning for anomaly detection. Performance study considering anomaly distribution in an imbalanced dataset," in *Proc. IEEE 5th Int. Conf. Cloud Comput. Artif. Intell.: Technol. Appl.*, 2020, pp. 1–8.
- [46] Z. Kang, H. Pan, S. C. Hoi, and Z. Xu, "Robust graph learning from noisy data," *IEEE Trans. Cybern.*, vol. 50, no. 5, pp. 1833–1843, May 2020.
- [47] R. Evans and E. Grefenstette, "Learning explanatory rules from noisy data," *J. Artif. Intell. Res.*, vol. 61, pp. 1–64, 2018.
- [48] Y. Kim, J. Yim, J. Yun, and J. Kim, "NLNL: Negative learning for noisy labels," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 101–110.
- [49] N. Moran, D. Schmidt, Y. Zhong, and P. Coady, "Noisier2Noise: Learning to denoise from unpaired noisy data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12 064–12 072.
- [50] J.-X. Zhong, N. Li, W. Kong, S. Liu, T. H. Li, and G. Li, "Graph convolutional label noise cleaner: Train a plug-and-play action classifier for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 1237–1246.
- [51] M. Z. Zaheer, J.-H. Lee, M. Astrid, A. Mahmood, and S.-I. Lee, "Cleaning label noise with clusters for minimally supervised anomaly detection," 2021, *arXiv:2104.14770*.
- [52] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [53] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset," *IEEE Access*, vol. 9, pp. 22 351–22 370, 2021.
- [54] A. F. Emmott, S. Das, T. Dietterich, A. Fern, and W.-K. Wong, "Systematic construction of anomaly detection benchmarks from real data," in *Proc. ACM SIGKDD Workshop Outlier Detection Description*, 2013, pp. 16–21.
- [55] C. R. Banbury et al., "Benchmarking tinyml systems: Challenges and direction," 2020, *arXiv:2003.04821*.
- [56] G. A. Noghre, A. D. Pazho, J. Sanchez, N. Hewitt, C. Neff, and H. Tabkhi, "ADG-Pose: Automated dataset generation for real-world human pose estimation," in *Proc. Int. Conf. Pattern Recognit. Artif. Intell.*, 2022, pp. 258–270.
- [57] G. Yu, Z. Cai, S. Wang, H. Chen, F. Liu, and A. Liu, "Unsupervised online anomaly detection with parameter adaptation for KPIs abrupt changes," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1294–1308, Sep. 2020.
- [58] W. Wang, Q. Chen, T. Liu, X. He, and L. Tang, "A distributed online learning approach to detect anomalies for virtualized network slicing," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.

- [59] A. Guezaz, Y. Asimi, M. Azrou, and A. Asimi, "Mathematical validation of proposed machine learning classifier for heterogeneous traffic and anomaly detection," *Big Data Mining Analytics*, vol. 4, no. 1, pp. 18–24, 2021.
- [60] S. Li, Y. Cheng, Y. Liu, W. Wang, and T. Chen, "Abnormal client behavior detection in federated learning," 2019, *arXiv:1910.09933*.
- [61] L. Erhan et al., "Smart anomaly detection in sensor systems: A multi-perspective review," *Inf. Fusion*, vol. 67, pp. 64–79, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253520303717>
- [62] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 315–324.
- [63] S. Bhatia, Y. Wang, B. Hooi, and T. Chakraborty, "Graphanogan: Detecting anomalous snapshots from attributed graphs," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2021, pp. 36–51.
- [64] Y. Zheng, M. Jin, Y. Liu, L. Chi, K. T. Phan, and Y.-P. P. Chen, "Generative and contrastive self-supervised learning for graph anomaly detection," *IEEE Trans. Knowl. Data Eng.*, early access, Oct. 12, 2021, doi: [10.1109/TKDE.2021.3119326](https://doi.org/10.1109/TKDE.2021.3119326).
- [65] Y. Liu, T. Dillon, W. Yu, W. Rahayu, and F. Mostafa, "Noise removal in the presence of significant anomalies for industrial IoT sensor data in manufacturing," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7084–7096, Aug. 2020.
- [66] Y. Tang et al., "Detection of magnetic anomaly signal based on information entropy of differential signal," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 4, pp. 512–516, Apr. 2018.
- [67] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6479–6488.
- [68] F. Muharemi, D. Logofătu, and F. Leon, "Machine learning approaches for anomaly detection of water quality on a real-world data set," *J. Inf. Telecommunication*, vol. 3, no. 3, pp. 294–307, 2019.
- [69] A. Castellani, S. Schmitt, and S. Squartini, "Real-world anomaly detection by using digital twin systems and weakly supervised learning," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4733–4742, Jul. 2021.
- [70] A. Ukil, S. Bandyopadhyay, C. Puri, and A. Pal, "IoT healthcare analytics: The importance of anomaly detection," in *Proc. IEEE 30th Int. Conf. Adv. Inf. Netw. Appl.*, 2016, pp. 994–997.
- [71] S. A. Haque, M. Rahman, and S. M. Aziz, "Sensor anomaly detection in wireless sensor networks for healthcare," *Sensors*, vol. 15, no. 4, pp. 8764–8786, 2015. [Online]. Available: <https://www.mdpi.com/1424-8220/15/4/8764>
- [72] X. Liu et al., "Privacy and security issues in deep learning: A survey," *IEEE Access*, vol. 9, pp. 4566–4593, 2020.
- [73] Y. Shen, G. Leus, and G. B. Giannakis, "Online graph-adaptive learning with scalability and privacy," *IEEE Trans. Signal Process.*, vol. 67, no. 9, pp. 2471–2483, May 2019.
- [74] B. Wang, J. Jia, and N. Z. Gong, "Graph-based security and privacy analytics via collective classification with joint weight learning and propagation," 2018, *arXiv:1812.01661*.
- [75] C. Yang, H. Wang, K. Zhang, L. Chen, and L. Sun, "Secure deep graph generation with link differential privacy," 2020, *arXiv:2005.00455*.
- [76] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *Proc. 17th EAI Int. Conf. Mobile Ubiquitous Systems: Comput. Netw. Serv.*, 2020, pp. 76–85.
- [77] J. Zhou et al., "Vertically federated graph neural network for privacy-preserving node classification," 2020, *arXiv:2005.11903*.
- [78] S. Sajadmanesh and D. Gatica-Perez, "Locally private graph neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2130–2145.
- [79] C. Zhang, S. Zhang, J. James, and S. Yu, "FASTGNN: A topological information protected federated learning approach for traffic speed forecasting," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8464–8474, Dec. 2021.
- [80] Q. Li, M. Coutino, G. Leus, and M. G. Christensen, "Privacy-preserving distributed graph filtering," in *Proc. IEEE 28th Eur. Signal Process. Conf.*, 2021, pp. 2155–2159.
- [81] T. Bakhshi and B. Ghita, "Anomaly detection in encrypted internet traffic using hybrid deep learning," *Secur. Commun. Netw.*, vol. 2021, pp. 1–6, 2021.
- [82] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, "MApGraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 1025–1038.
- [83] Y. Wu, H.-N. Dai, and H. Tang, "Graph neural networks for anomaly detection in industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9214–9231, Jun. 2022.
- [84] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [85] A. Protopogerou, S. Papadopoulos, A. Drosou, D. Tzovaras, and I. Refanidis, "A graph neural network method for distributed anomaly detection in IoT," *Evolving Syst.*, vol. 12, no. 1, pp. 19–36, 2021.
- [86] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [87] R. Zhou, Q. Zhang, P. Zhang, L. Niu, and X. Lin, "Anomaly detection in dynamic attributed networks," *Neural Comput. Appl.*, vol. 33, no. 6, pp. 2125–2136, 2021.
- [88] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4419–4425.
- [89] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decis. Support Syst.*, vol. 133, 2020, Art. no. 113303.
- [90] M. Salehi and L. Rashidi, "A survey on anomaly detection in evolving data: [with application to forest fire risk prediction]," *ACM SIGKDD Explorations Newsl.*, vol. 20, no. 1, pp. 13–23, 2018.
- [91] K. Ding, J. Li, and H. Liu, "Interactive anomaly detection on attributed networks," in *Proc. 12th ACM Int. Conf. Web Search Data Mining*, 2019, pp. 357–365.
- [92] G. Xue, M. Zhong, J. Li, J. Chen, C. Zhai, and R. Kong, "Dynamic network embedding survey," *Neurocomputing*, vol. 472, pp. 212–223, 2022.
- [93] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surveys*, vol. 55, no. 3, pp. 1–39, 2022.
- [94] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4027–4035.
- [95] H. Wang et al., "Groot: An event-graph-based approach for root cause analysis in industrial settings," in *Proc. IEEE/ACM 36th Int. Conf. Automated Softw. Eng.*, 2021, pp. 419–429.
- [96] K. Ding, K. Shu, X. Shan, J. Li, and H. Liu, "Cross-domain graph anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2406–2415, Jun. 2022.
- [97] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by back-propagation," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.
- [98] J. Hoffman et al., "LSDA: Large scale detection through adaptation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014.
- [99] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *J. Mach. Learn. Res.*, vol. 12, pp. 2493–2537, 2011.
- [100] Q. Li, "Literature survey: Domain adaptation algorithms for natural language processing," 2012.
- [101] A. Søgaard, "Semi-supervised learning and domain adaptation in natural language processing," *Synth. Lectures Hum. Lang. Technol.*, vol. 6, no. 2, pp. 1–103, 2013.
- [102] Y. Zhang et al., "Collaborative unsupervised domain adaptation for medical image diagnosis," *IEEE Trans. Image Process.*, vol. 29, pp. 7834–7844, 2020.
- [103] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DioT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, Dallas, TX, USA, 2019, pp. 756–767.
- [104] X. Ni, X. Xu, L. Lyu, C. Meng, and W. Wang, "A vertical federated learning framework for graph convolutional network," 2021, *arXiv:2106.11593*.
- [105] A. Borghesi, A. Bartolini, M. Lombardi, M. Milano, and L. Benini, "Anomaly detection using autoencoders in high performance computing systems," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 9428–9433.
- [106] Z. Chen, C. K. Yeo, B. S. Lee, and C. T. Lau, "Autoencoder-based network anomaly detection," in *Proc. IEEE Wireless Telecommun. Symp.*, 2018, pp. 1–5.
- [107] S. Park, M. Kim, and S. Lee, "Anomaly detection for HTTP using convolutional autoencoders," *IEEE Access*, vol. 6, pp. 70 884–70 901, 2018.

- [108] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2017, pp. 665–674, doi: [10.1145/3097983.3098052](https://doi.org/10.1145/3097983.3098052).
- [109] L. Beggel, M. Pfeiffer, and B. Bischl, "Robust anomaly detection in images using adversarial autoencoders," in *Machine Learning and Knowledge Discovery in Databases*, U. Brefeld, E. Fromont, A. Hotho, A. Knobbe, M. Maathuis, and C. Robardet Eds., Cham, Switzerland: Springer, 2020, pp. 206–222.
- [110] O. Atkinson, A. Bhardwaj, C. Englert, V. S. Ngairangbam, and M. Spannowsky, "Anomaly detection with convolutional graph neural networks," *J. High Energy Phys.*, vol. 2021, no. 8, 2021, Art. no. 80, doi: [10.1007/JHEP08\(2021\)080](https://doi.org/10.1007/JHEP08(2021)080).
- [111] S. Bandyopadhyay, S. V. Vivek, and M. Murty, "Outlier resistant unsupervised deep architectures for attributed network embedding," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 25–33.
- [112] X. Teng, M. Yan, A. M. Ertugrul, and Y.-R. Lin, "Deep into hypersphere: Robust and unsupervised anomaly discovery in dynamic networks," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018.
- [113] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2014, pp. 701–710, doi: [10.1145/2623330.2623732](https://doi.org/10.1145/2623330.2623732).
- [114] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2016, pp. 855–864, doi: [10.1145/2939672.2939754](https://doi.org/10.1145/2939672.2939754).
- [115] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077, doi: [10.1145/2736277.2741093](https://doi.org/10.1145/2736277.2741093).
- [116] C. Yang, Z. Liu, D. Zhao, M. Sun, and E. Chang, "Network representation learning with rich text information," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, 2015, pp. 2111–2117.
- [117] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA: Association for Computing Machinery, 2018, pp. 2672–2681, doi: [10.1145/3219819.3220024](https://doi.org/10.1145/3219819.3220024).
- [118] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4419–4425, doi: [10.24963/jicai.2019/614](https://doi.org/10.24963/jicai.2019/614).
- [119] S. Feng, Z. Tan, R. Li, and M. Luo, "Heterogeneity-aware twitter bot detection with relational graph transformers," 2021, *arXiv:2109.02927*.
- [120] Y. Liu et al., "Anomaly detection in dynamic graphs via transformer," *IEEE Trans. Knowl. Data Eng.*, early access, Nov. 2, 2021, doi: [10.1109/TKDE.2021.3124061](https://doi.org/10.1109/TKDE.2021.3124061).
- [121] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*.
- [122] H. E. Egilmez and A. Ortega, "Spectral anomaly detection using graph-based filtering for wireless sensor networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2014, pp. 1085–1089.
- [123] B. Chen et al., "Gccad: Graph contrastive coding for anomaly detection," 2021, *arXiv:2108.07516*.
- [124] M. Aledhari, R. Razzak, R. M. Parizi, and F. Saeed, "Federated learning: A survey on enabling technologies, protocols, and applications," *IEEE Access*, vol. 8, pp. 140 699–140 725, 2020.
- [125] K. Bonawitz et al., "Towards federated learning at scale: System design," *Proc. Mach. Learn. Syst.*, vol. 1, pp. 374–388, 2019.
- [126] Y. Chen, Y. Ning, Z. Chai, and H. Rangwala, "Federated multi-task hierarchical attention model for sensor analytics," in *Proc. Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8, doi: [10.1109/IJCNN48605.2020.9207508](https://doi.org/10.1109/IJCNN48605.2020.9207508).
- [127] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," in *Proc. NIPS Workshop Private Multi-Party Mach. Learn.*, 2016. [Online]. Available: <https://arxiv.org/abs/1610.05492>
- [128] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazani, "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=BkluqISFDS>
- [129] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2017, pp. 1175–1191, doi: [10.1145/3133956.3133982](https://doi.org/10.1145/3133956.3133982).
- [130] R. Chandra and S. Cripps, "Bayesian multi-task learning for dynamic time series prediction," in *Proc. Int. Joint Conf. Neural Netw.*, 2018, pp. 1–8.
- [131] S. Li, Y. Song, and G. Zhou, "Leak detection of water distribution pipeline subject to failure of socket joint based on acoustic emission and pattern recognition," *Measurement*, vol. 115, pp. 39–44, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0263224117306498>
- [132] L.-J. Chen, Y.-H. Ho, H.-H. Hsieh, S.-T. Huang, H.-C. Lee, and S. Mahajan, "ADF: An anomaly detection framework for large-scale PM2.5 sensing systems," *IEEE Internet Things J.*, vol. 5, no. 2, pp. 559–570, Apr. 2018.
- [133] D. L. Goodhue and D. W. Straub, "Security concerns of system users: A study of perceptions of the adequacy of security," *Inf. Manage.*, vol. 20, no. 1, pp. 13–27, Jan. 1991, doi: [10.1016/0378-7206\(91\)90024-V](https://doi.org/10.1016/0378-7206(91)90024-V).
- [134] R. A. Sater and A. B. Hamza, "A federated learning approach to anomaly detection in smart buildings," *ACM Trans. Internet Things*, vol. 2, no. 4, Aug. 2021, doi: [10.1145/3467981](https://doi.org/10.1145/3467981).
- [135] H. Zhang, T. Shen, F. Wu, M. Yin, H. Yang, and C. Wu, "Federated graph learning—a position paper," 2021, *arXiv:2105.11099*.
- [136] E. J. Candès, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis?," *J. ACM*, vol. 58, no. 3, Jun. 2011, doi: [10.1145/1970392.1970395](https://doi.org/10.1145/1970392.1970395).
- [137] D. L. Donoho, "For most large underdetermined systems of linear equations the minimal l_1 -norm solution is also the sparsest solution," *Commun. Pure Appl. Math.*, vol. 59, pp. 797–829, 2006.
- [138] R. Paffenroth, P. du Toit, R. Nong, L. Scharf, A. P. Jayasumana, and V. Bandara, "Space-time signal processing for distributed pattern detection in sensor networks," *IEEE J. Sel. Topics Signal Process.*, vol. 7, no. 1, pp. 38–49, Feb. 2013.
- [139] A. Makhzani, J. Shlens, N. Jaitly, and I. Goodfellow, "Adversarial autoencoders," in *Proc. Int. Conf. Learn. Representations*, 2016. [Online]. Available: <http://arxiv.org/abs/1511.05644>
- [140] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [141] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim, "Graph transformer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32.
- [142] J. Klicpera, S. Weissenberger, and S. Günnemann, "Diffusion improves graph learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.
- [143] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 4116–4126.
- [144] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," 2016, *arXiv:1611.01144*.
- [145] A. Ortega, P. Frossard, J. Kováčević, J. M. F. Moura, and P. Vandergheynst, "Graph signal processing: Overview, challenges, and applications," in *Proc. IEEE*, vol. 106, no. 5, pp. 808–828, May 2018.
- [146] A. Sandryhaila and J. M. F. Moura, "Discrete signal processing on graphs: Frequency analysis," *IEEE Trans. Signal Process.*, vol. 62, no. 12, pp. 3042–3054, Jun. 2014.
- [147] G. Shen and A. Ortega, "Joint routing and 2D transform optimization for irregular sensor network grids using wavelet lifting," in *Proc. Int. Conf. Inf. Process. Sensor Netw.*, 2008, pp. 183–194.
- [148] X. Zhu and M. Rabbat, "Graph spectral compressed sensing for sensor networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2012, pp. 2865–2868.
- [149] M. Zheng, C. Zhou, J. Wu, S. Pan, J. Shi, and L. Guo, "FraudNE: A joint embedding approach for fraud detection," in *Proc. Int. Joint Conf. Neural Netw.*, 2018, pp. 1–8.
- [150] Y. Dou, K. Shu, C. Xia, P. S. Yu, and L. Sun, "User preference-aware fake news detection," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 2051–2055.
- [151] N. Liu, X. Huang, and X. Hu, "Accelerated local anomaly detection via resolving attributed networks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 2337–2343.
- [152] L. Zhang, J. Yuan, Z. Liu, Y. Pei, and L. Wang, "A robust embedding method for anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.
- [153] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 594–602.
- [154] X. Wang, B. Jin, Y. Du, P. Cui, Y. Tan, and Y. Yang, "One-class graph neural networks for anomaly detection in attributed networks," *Neural Comput. Appl.*, vol. 33, no. 18, pp. 12 073–12 085, 2021.

- [155] Z. Peng, M. Luo, J. Li, L. Xue, and Q. Zheng, "A deep multi-view framework for anomaly detection on attributed networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 6, pp. 2539–2552, Jun. 2022.
- [156] Y. Li, X. Huang, J. Li, M. Du, and N. Zou, "SpecAE: Spectral autoencoder for anomaly detection in attributed networks," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2233–2236.
- [157] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Computer Vision—ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars Eds., Cham, Switzerland: Springer, 2014, pp. 740–755.



Armin Danesh Pazho (Student Member, IEEE) is currently working toward the PhD degree with the University of North Carolina at Charlotte, NC, USA. With a focus on Artificial Intelligence, Computer Vision, and Deep Learning, his research delves into the realm of developing AI for practical, real-world applications and addressing the challenges and requirements inherent in these fields. Specifically, his research covers action recognition, anomaly detection, person re-identification, human pose estimation, and path prediction.



Ghazal Alinezhad Noghre (Student Member, IEEE) is currently working toward the PhD degree in electrical and computer engineering with the University of North Carolina at Charlotte, NC, USA. Her research interests include concentrates on artificial intelligence, machine learning, and computer vision. She is particularly interested in the applications of anomaly detection, action recognition, and path prediction in real-world environments, and the challenges associated with these fields.



Arnab A Purkayastha received the PhD degree from the University of North Carolina at Charlotte, in 2021. He is an assistant professor with the Electrical and Computer Engineering Department, Western New England University, Massachusetts. His research interests and activities lie in the recent advances in high performance computing and machine learning fields, including system level integration both at the cloud and at the edge.



Jagannadh Vempati received the PhD degree in computer science and engineering from the University of North Texas. He is a research scientist with Siemens Technology in Princeton – NJ performing research and development on designing cyber defense solutions that incorporate domain know-how and the semantic context of attack detection and response solutions. His responsibilities include developing new methods and techniques that improve the value of security analytics in Cyber Defense by integrating industrial domain expertise and semantics/context and guiding Siemens Business Units to include research results into products, solutions, and services. His research is focused on data-driven cyber security and designing resilient networks. His research also focuses on designing, developing, and optimizing anomaly detection models using artificial intelligence.



Martin Otto received the PhD degree in computer science from Paderborn University, Paderborn, Germany, in 2005, working on fault attack side channels on smart cards. He is a researcher and research manager with Siemens Technology, Siemens' central R&D organization, since 2005. He is currently the head of the research group "Cybersecurity Service Innovation" at Siemens Corporation, Siemens Technology, in Princeton, NJ, USA. His mission is to provide Siemens business units with technology solutions and innovations that enable Siemens to provide state of the security services to customers. A specific focus is on helping Siemens customers that operate energy systems and other parts of national critical infrastructure to detect, react to, mitigate, and otherwise defend against cyber attacks. He held positions in the US, Canada, and Germany, among them as global Head of the Siemens CERT (Computer Emergency Response Team).



Hamed Tabkhi (Member, IEEE) received the PhD degree in computer engineering from Northeastern University, in 2014. He is the Associate Professor of computer engineering with the University of North Carolina Charlotte (UNCC). His research and scholarship activities focus on transformative computer system solutions to bring recent advances in Artificial Intelligence (AI) to address real-world problems. In particular, he focuses on AI-based solutions to enhance our communities' safety, health, and overall well-being.